

Kristálytiszta Elektronika 2

A Kristálytiszta Elektronika tananyag fejlesztését az Európai Bizottság támogatta az Erasmus+ program keretén belül „2020-1-HU01-KA201-078779” számú „Elektronika és programozás témájú innovatív tananyag fejlesztése a köznevelés számára” című projekt kapcsán.

Az Európai Unió
Erasmus+ programjának
társfinanszírozásával



A projektet az alábbi 4 intézményből álló nemzetközi partnerség valósította meg:

- Xtalin Mérnöki Tervező Kft. – Budapest
- ELTE Bolyai János Gyakorló Általános Iskola és Gimnázium – Szombathely
- Bolyai Farkas Elméleti Líceum – Târgu Mureș
- Pro Ratio Alapítvánnyal karöltve működő Madách Imre Gimnázium – Šamorín



XTALIN



Szerzői jogok:

Jelen tananyag az Xtalin Mérnöki Tervező Kft., mint koordinátor által vezetett partnerség szellemi terméke. Az anyagok oktatási felhasználásra készültek, ezért e célra szabadon felhasználhatóak; tartalmuk azonban nem módosítható, továbbfejleszthető az Xtalin Kft. írásban adott engedélye nélkül. Az anyagok újraközlése tartalmilag változatlan formában, kizárólag a tananyag szerzőinek és az eredeti tananyag elérhetőségének világos megjelölésével lehetséges, kizárólag az Xtalin Kft. írásban adott engedélyével.

Kapcsolat: <http://kristalytisztaelektronika2.hu/>
info@kristalytisztaelektronika.hu

7. A mikrokontroller I/O portjai

Írta: Csököly István Kristóf

Lektorálta: Szabó Ádám

Kedves Olvasó, ebben a tananyagrészben szeretném neked bemutatni a mikrovezérlő ki- és bemeneti egységeit, ezek funkcióját és felépítését. Megismerheted, hogyan kapcsolódik a vezérlő a külvilághoz, pontosan mit is jelent az I/O (Input/Output) port kifejezés. Vázlatosan bemutatok néhány kommunikációs lehetőséget, illetve szót ejtek többek között az analóg és digitális világ kapcsolatáról. Ez utóbbiról a későbbi fejezetekben részletes információkkal szolgálunk, azonban fontosnak tartom, hogy kialakuljon számodra egy átfogó kép.

A fejezet nem lenne teljes, ha nem mutatnám meg, hogyan tudod magad is beállítani a ki- és bemeneteket a vezérlőn. Szerencsére előre elkészített példakódokon keresztül mindent megmutatok, amire kezdetben szükséged lehet. A későbbiekben erre a tudásra szépen, fokozatosan építhetsz. Remélem, sok hasznos újdonsággal szolgálhatok és bízom benne, hogy ez a tananyagrész is élvezetes lesz számodra.

Vágjunk is bele! :)

A MIKROVEZÉRLŐ LÁBAI

Mit is értünk a vezérlő lábán?

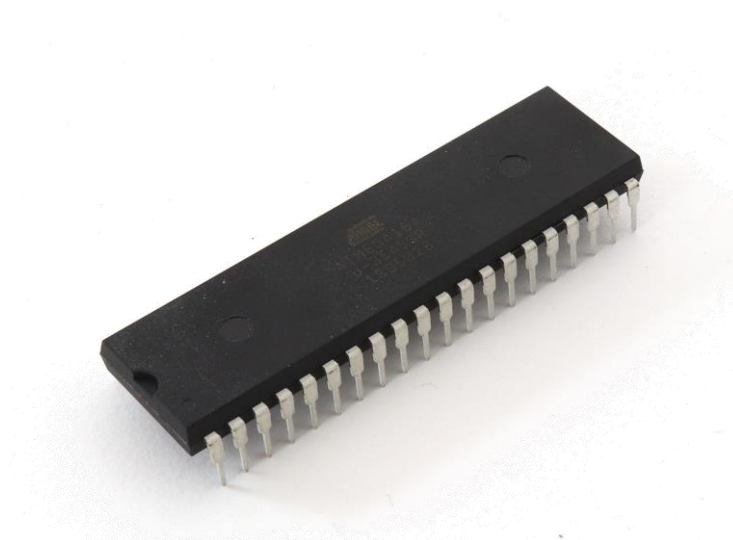
A mikrovezérlő lábai azok a pontok, ahol az eszköz belseje kapcsolatot képes kialakítani a külvilággal. A vezérlő belsejéből a számunkra szükséges jeleket fémezett kivezetések, más néven lábak segítségével tudjuk hozzáférhetővé tenni.

Ezek a kivezetések kettős funkcióval rendelkeznek. Egyrészt az elektromos (galvanikus) kapcsolat kialakítása, másrészt a chip fizikai rögzítése a feladatuk (ugyanis a lábakat forrasztjuk hozzá az áramköri panelhez).

Manapság ugyanaz a vezérlő többféle tokozással is elérhető, különböző tokozásokhoz különböző kivezetéseket alkalmaznak a gyártók.

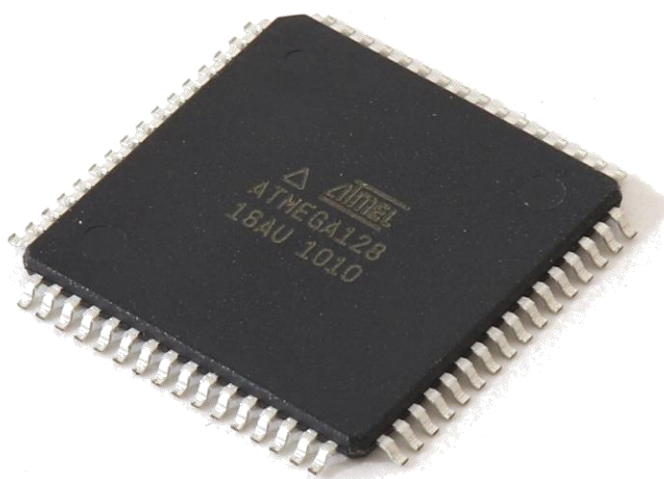
A teljesség igénye nélkül ismerjük meg a leggyakrabban alkalmazott megoldásokat:

DIP (Dual in-line package): Az eszköz két (általában hosszabb) oldalán kerülnek elhelyezésre a kivezetések. Furatszerelt alkalmazásokban ez a legelterjedtebb megoldás.



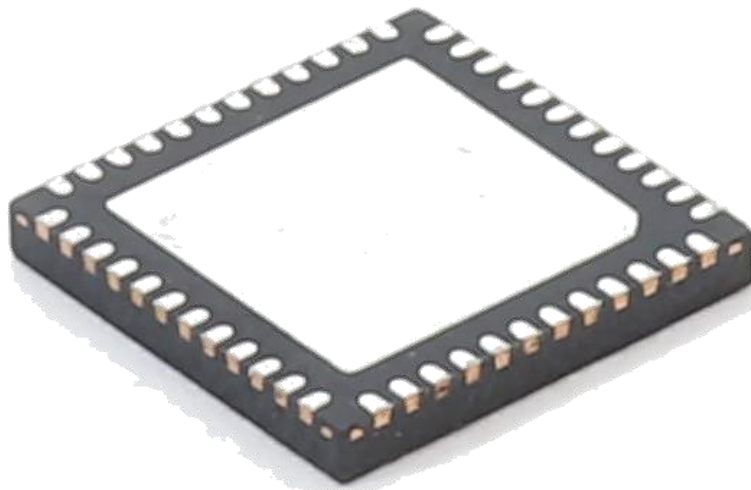
1. ábra: DIP tokozású eszköz

QFP (Quad Flat Package): Az eszköz mind a négy oldalán elhelyezésre kerülnek a kivezetések. Felületszerelt alkalmazásokban elterjedt. A DIP tokozáshoz képest a lábak sokkal közelebb helyezkednek el egymáshoz.



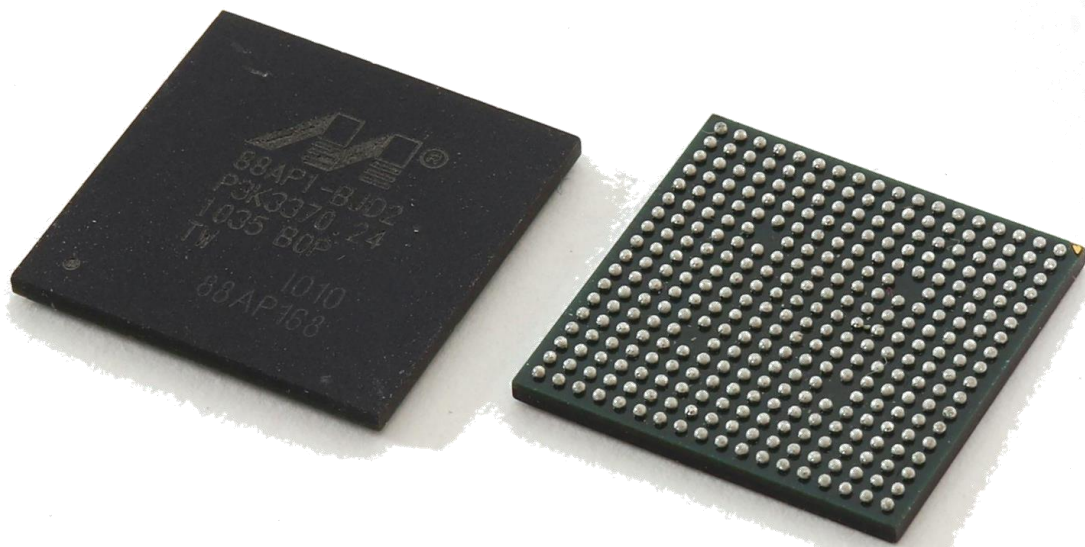
2. ábra: QFP tokozású eszköz

QFN (Quad Flat No Lead): Ez a tokozás a QFP-hez hasonlít annyi különbséggel, hogy a lábak helyett a tok szélén és alján forrpad-ek találhatók.



3. ábra: QFN tokozás alulnézetből

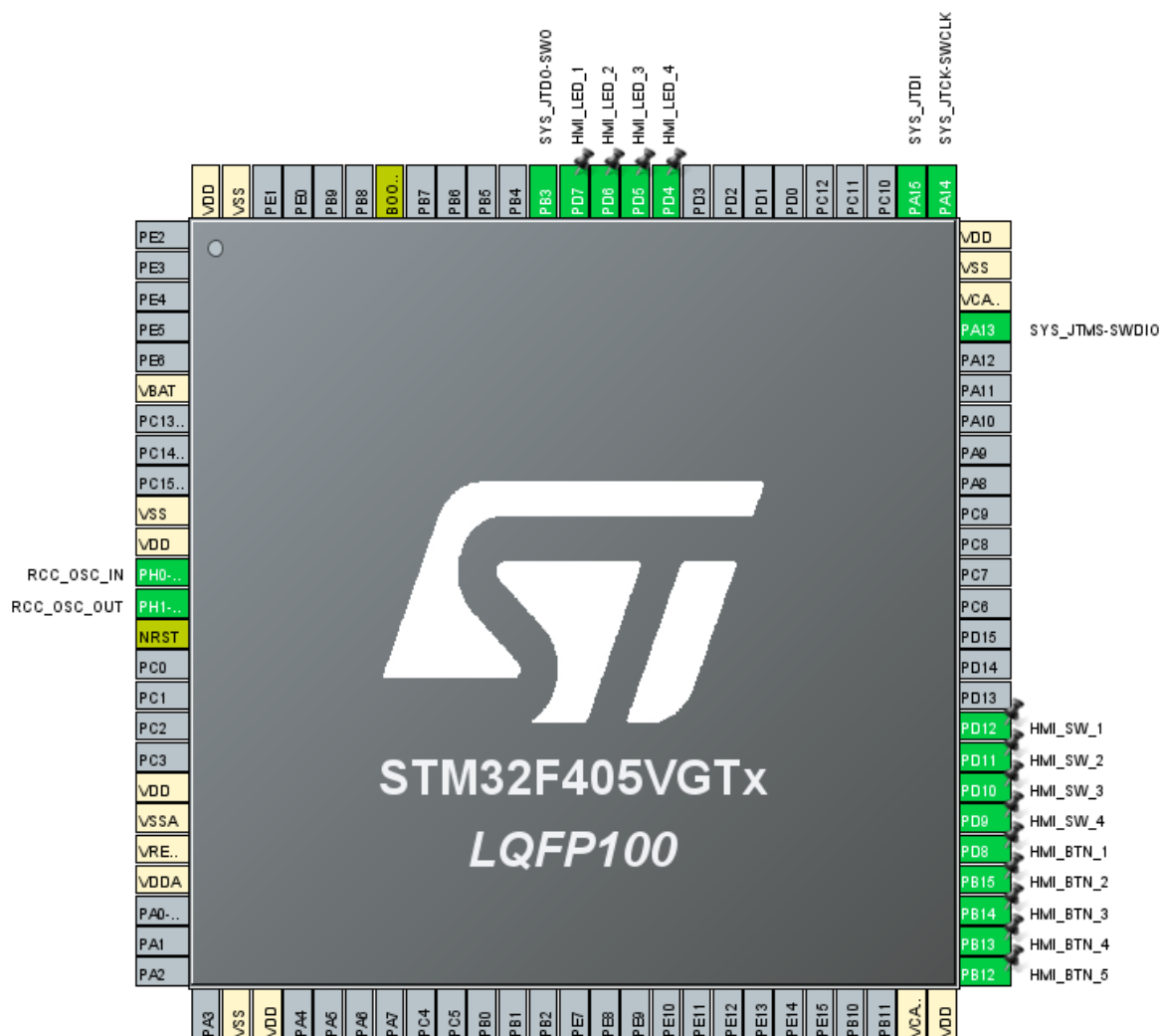
BGA (Ball Grid Array): Az eszköz alján négyzetháló mentén találhatók síkban apró forrpad-ek, amiket a gyártó előre leónoz. Ezekről az óngömbökről kapta a nevét a tokozás.



4. ábra: BGA tokozás felül- és alulnézetből

Napjainkban egy nagyobb teljesítményű, sok funkció ellátására képes mikrovezérlő rengeteg lábbal kapcsolódhat a külvilághoz. Első ránézésre nem egyszerű feladat eldönteni, melyik láb pontosan milyen funkciót valósít meg.

A továbbiakban az STMicroelectronics STM32F4 mikrovezérlőn keresztül mutatom be a lábak típusait, illetve ezek fizikai kialakítását. A későbbi tananyagrészekben is ezen eszközhöz kapcsolódóan lesz lehetőséged ismereteidet bővíteni. A mi STM32F4-es mikrokontrollerünk LQFP (Low-profile Quad Flat Package) 100 tokozással készült, ahol a 100-as érték a lábak számát jelenti.



5. ábra: STM32F405 vezérlő lábkiosztása

MŰKÖDÉSHEZ ELENEDHETETLEN LÁBAK

TÁPLÁBAK

Vss, Vssa, Vee, GND, AGND:

Ezeket a lábakat rendszerint az áramkör földpotenciáljához kapcsoljuk. A földpotenciál szerepe az áramkörben, hogy minden más feszültségszintet ehhez képest adunk meg és így ezt a szintet vehetjük nullának.

V_{dd} , V_{dda} , V_{bat} , V_{cc} , V_{in} :

Ezekre a táplálbakra az adott alkatrész adatlapjában meghatározott tartománynak megfelelő tápfeszültséget kötünk, aminek következtében a mikrovezérlő életre kel, és azonnal nekiáll feladata elvégzésének.

REFERENCIA FESZÜLTÉG LÁBAK

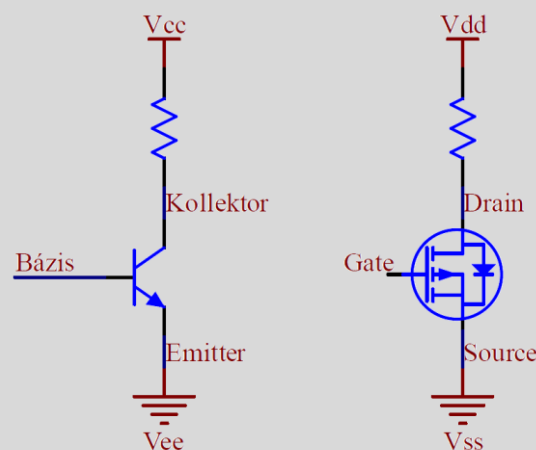
A kontrolleren található V_{ref+} és A_{ref} lábakra úgynevezett referenciafeszültséget kapcsolhatunk. Ez a feszültség többféleképpen is előállítható: a tápfeszültség bekötésével, tápfeszültség kondicionálása által, dedikált egység által.

A mikrovezérlő ezeket a feszültségeket az analóg-digitális átalakító (ADC) és digitális-analóg átalakító (DAC) periféria referenciájaként használja fel.

Az analóg-digitális és digitális-analóg átalakítókkal igazán részletesen a 11. és a 12. tananyagrészen ismerkedhetünk meg.

Kitekintés:

A fenti elnevezések első ránézésre elég zavaróak lehetnek, de nézzük mi rejtőzik a sok jelölés mögött.



6. ábra: Táplálbak jelölése

A V_{cc} -ben a "c" jelölés a bipoláris NPN tranzisztor "collector"-ára, V_{dd} esetén a "d" pedig a N-MOSFET "drain"-jére utal. Ezek kapcsolódnak (ellenálláson keresztül) a pozitív tápfeszültséghez.

A V_{ee} -ben az "e" a bipoláris NPN tranzisztor "emitter"-ére, a V_{ss} -ben pedig az "s" az N-MOSFET "source"-ára utal.

Az AGND, a V_{ssa} és a V_{dda} esetén az "A" jelölés az analóg elektronikára utal. A mikrovezérlőben külön áramút ("vezetékvezetés") található az analóg és digitális egységek energiával történő ellátására. Ezáltal lehetőségünk van a kontrolleren belül található analóg és digitális eszközeinket egymástól független tápáramkörre kapcsolni. Erre azért lehet szükség, mert az analóg eszközök pontossága nagyban függ a tápfeszültségen megjelenő zavarok mértékétől.

V_{bat} : az akkumulátor (battery) pozitív feszültségű kapcsát csatlakoztathatjuk ehhez a lábhoz.

Akkumulátor alatt ne valami hatalmas dologra gondoljunk. A legtöbb esetben ez egy egyszerű 3 V-os gombelem vagy hasonló feszültségű akkumulátor, esetleg kondenzátor. Az akkumulátor ez esetben a biztonsági mentést és néhány kritikus funkciót lát el energiával (RealTimeClock, backup registers, backup SRAM).

Amikor egy bonyolultabb elektronikát tervezünk, az feltehetően több alegységből fog állni. Lehetnek benne analóg jelek mérésére szolgáló részek, kisáramú digitális kimenetek, de lehetnek benne olyan részek is, melyek nagy teljesítményigényűek, ráadásul az áramfelvételük időben gyorsan változik. Ilyenek például az elektronika többi részéhez képest nagy áramot fogyasztó, nagy fényerejű LED-ek, melyek gyors PWM-mel (lásd később) vannak kapcsolgatva, illetve a nagy teljesítményfelvételű vezeték nélküli kommunikációs modulok stb.

A Kristálytisza Elektronika 2 fejlesztőpanel kapcsolási rajzán kutakodva például észrevehetjük, hogy 12 V-os tápbemenetről állít elő 5 V-ot egy kapcsolóüzemű tápegység. Az 5 V-ról PWM-mel van meghajtva a DC motor, ezért, ha oszcilloszkóppal megvizsgálunk az 5 V-os tápfeszültséget, elképzelhető, hogy zajos lenne. Az áramkörön az 5 V-ból egy újabb, alacsony zajú kapcsolóüzemű tápegységgel állítjuk elő a 3.3 V-os tápfeszültséget (melyet gyakran 3V3-ként jelölünk), amiből további szűrés után megkapjuk az ADC-hez szükséges V_{dda} és V_{ref+} jeleket. Meg kell jegyezni, hogy a V_{dda} és V_{ref+} jelek így ugyan stabilak, de nem feltétlenül pontosan 3V3 az értékük, mivel az adatlapja szerint a 3V3-as táp IC is rendelkezik +/-3% pontatlansággal. Ha bárhol ennél pontosabb AD-konverzióra lett volna szükségünk, akkor a V_{ref+} lábra egy pontosabb referenciafeszültséget kellett volna kötni, például egy 3V0, vagy 2V5 feszültségű, 1% hibatűrésű referenciafeszültség IC kimenetét.

A fentiek alapján látható, hogy a táprendszer megfelelő kialakítása és méretezése mennyire fontos feladat. Amennyiben szeretnél mélyebben is elmerülni ebben a témakörben a Kristálytisza Elektronika 7. fejezetét (Kapcsolóeszközök) bátran ajánlom.

ÓRAJEL

Minden mikrovezérlőnek szüksége van órajelre (clock / oscillator) a program végrehajtásához. Vannak olyan vezérlők, amelyek belső vagy külső órajelet használnak és akadnak olyan megoldások is, ahol a mikrokontroller úgy van tervezve, hogy választani lehessen mind a két opció közül. Külső órajel esetén a vezérlőn dedikált lábakat jelölnek ki erre a célra, ide csatlakoztathatjuk a kristályoszillátort, amely biztosítja a mikrovezérlő működéséhez nélkülözhetetlen órajelet.

Az STM32F4 vezérlőn ilyen például a PH0 és a PH1 láb, amelyet kizárólag a kristályoszillátor bekötésére használunk (jelen tananyag rész keretein belül).

RESET

Ez a láb szolgál a vezérlő (tápellátás nélküli) alapállapotba állítására. Általában egy adott hosszúságú, alacsony logikai értékű jelet kell rákapcsolni.

PROGRAMOZÁSHOZ HASZNÁLT LÁBAK

Ha már van egy mikrovezérlőnk és szeretnénk is vezérelni valamit, akkor a programot fel kell töltenünk a vezérlő memóriájába, amit ebben a tananyagrészen a JTAG interfésszel teszünk meg. Erre külön kijelölt lábakon van lehetőségünk:

- PB03: SYS_JTDO-SWO (adatkimenet)
- PA15: SYS_JTDI (adatbemenet)
- PA14: SYS_JTCK-SWCLK (órajel)
- PA13: SYS_JTMS-SWDIO (módválasztás)

Másik lehetőségünk a felprogramozásra az SWD (Serial Wire Debug) interfész:

- PB03: SYS_JTDO-SWO (opcionális jel, amelyen a mikrovezérlő küldhet adatot hibakeresés közben)
- PA14: SYS_JTCK-SWCLK (órajel)
- PA13: SYS_JTMS-SWDIO (adat be- és kimenet)

ÁLTALÁNOSAN FELHASZNÁLHATÓ KI- ÉS BEMENETEK

GPIO (General-Purpose Input/Output) lábak konfigurálhatók logikai ki- és bemenetként, amelyeket a programunkból állíthatunk, illetve olvashatunk, valamint speciális funkciókat is elláthatnak (analóg ki- és bemenetek, kommunikációs interfészek stb.). A vezérlőn található lábakat csoportokba rendezik, ezeket portoknak nevezzük (PA-tól PE-ig). Ezekhez a portokhoz egy-egy regiszter tartozik, így programozáskor az adott lábra a port regiszterének megfelelő bitjével hivatkozhatunk.

HARDVERES MEGSZAKÍTÁSOK

Néhány GPIO lábon beállíthatjuk, hogy különböző események fel- vagy lefutó él, alacsony vagy magas logikai szint esetén a kontrollerben megszakítást generáljon.

A programmegszakításokról (Interrupt) részletesen olvashatsz a Kristálytisza Elektronika első részének Megszakítások a szoftverben - interruptok tananyagréssében. Én most csak egy rövid áttekintést szeretnék adni ezen funkció lehetőségeiről.

Az interrupt vagy más néven megszakítás lényege, hogy valamilyen esemény hatására a program futása megszakad. Ezt követően az adott megszakításhoz rendelt, szoftverben megvalósított megszakításkezelő függvény (interrupt handler) fut le. A vezérlő ezután visszatér a megszakított program végrehajtásához. Pontosan ott folytatódik a végrehajtás, ahol a megszakítás következtében abbamaradt.

Kitekintés:

Megszakítások

Megszakítás sok helyről érkezhetsz: digitális bemenet, timer, ADC, kommunikációs perifériák, programvezérelt módon (rendszerhívás), programhibák (0-val való osztás), véletlenszerű hibák

(memóriahiba). Ezeket két csoportba oszthatjuk: hardveres és szoftveres megszakításokra.

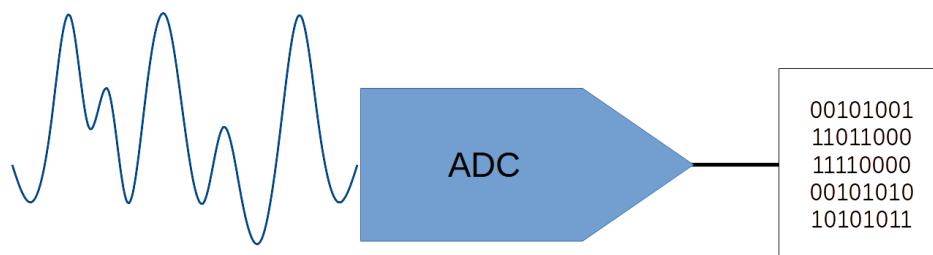
Hardveres interruptok segítségével gyorsan lehet reagálni a kiváltó eseményekre, ezért is szokták úgy emlegetni az interruptot, hogy "Függvényhívás gombnyomásra", de fontos, biztonságkritikus kódrészleteket is le lehet így futtatni egy veszélyes esemény bekövetkeztekor.

Belátható, hogy nem lehet minden megszakítás egyformán fontos. Ha egyszerre több interrupt érkezik be, valahogyan választanunk kell, melyiket szolgáljuk ki először. Erre ad megoldást az interrupt priority level (megszakítás fontossági szint), amely szerint a megszakítások fontosságuk szerint sorba rendezhetők.

Előfordulhat olyan kialakítás is, ahol maguk az interrupt rutinok is megszakíthatók (preempció), ezzel garantálva, hogy mindig a legfontosabb kód fut. Ez a szemlélet azonban hordozhat veszélyt is (ha magas prioritású megszakítás gyakran fut, előfordulhat, hogy az alacsonyabb szintűek nem tudnak futni, ezt hívjuk kiéheztetésnek).

A megszakítás érvényre jutásához szükséges engedélyezni az adott perifériához tartozó és a globális interruptot is. Amikor megszakítás generálódik, egy a megszakításokhoz tartozó regiszter megfelelő bitje beállítódik (bebillen az interrupt flag), kiszolgálás után a flaget törölni kell, különben tévesen ismét jelzést kapunk.

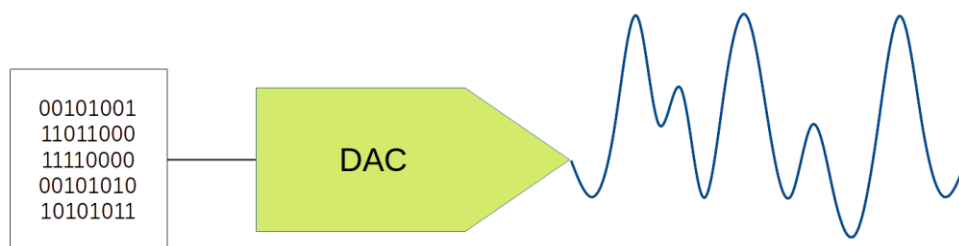
ANALÓG BEMENETEK



7. ábra: Analóg-digitális átalakító

Vannak olyan vezérlők (például az általunk is használt STM32F4 széria), melyek rendelkeznek analóg-digitális átalakítóval (Analog to Digital Converter, ADC). Ez olyan feszültségjelek pontos vizsgálatára ad lehetőséget, melyek nemcsak két logikai állapottal rendelkeznek (alacsony feszültség szint, magas feszültség szint), hanem bármilyen feszültségértéket felvehetnek egy adott tartományon belül. (Amennyiben másfajta jelet szeretnénk mérni, például nyomást, vagy hőmérsékletet, akkor először valamilyen külső áramkörrel át kell alakítanunk feszültségjellé, amit aztán képesek leszünk az ADC-vel mérni. Erről a 13. tananyagrészben bővebben is beszámolunk.)

ANALÓG KIMENETEK

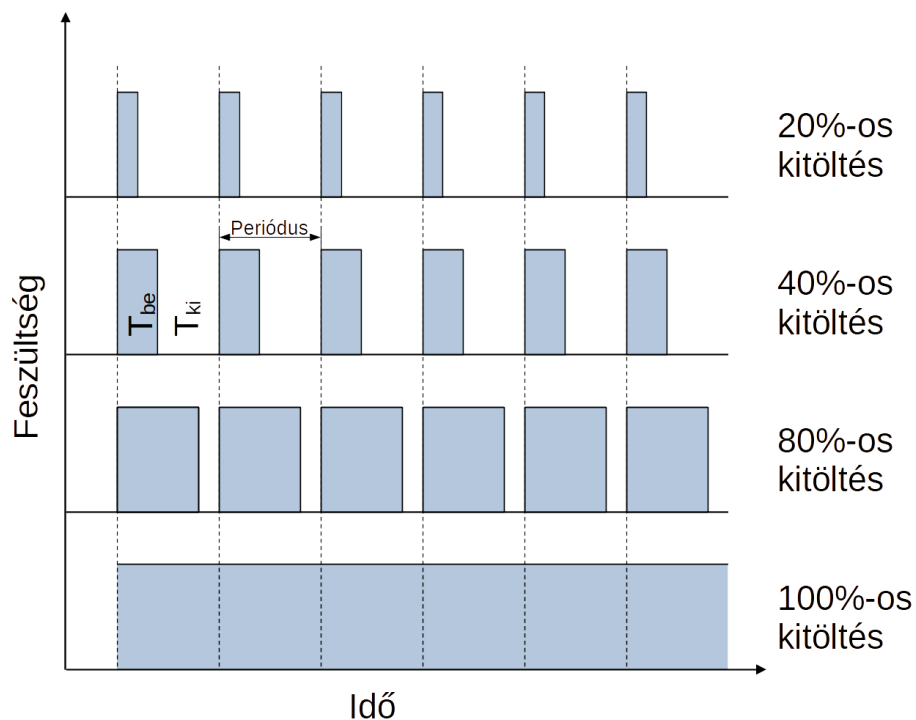


8. ábra: Digitális-analóg átalakító

Az ADC-hez hasonlóan elterjed a digitális-analóg átalakítók (Digital to Analog Converter, DAC) használata is. Az adott lábon olyan kimeneti feszültséggel állítható elő, mely számos, akár 256, 1024, 4096, vagy még ennél is több különböző jelszinttel rendelkezik. Az ADC és DAC perifériákról egy későbbi tananyagrészen fogunk részletesebben beszélni.

IMPULZUSSZÉLESSÉG-MODULÁCIÓ

Bizonyos kimeneten hardveres támogatással (így nagy pontossággal) létrehozható olyan négyszögjel, melynek állítható mind a periódusideje, mind a kitöltési tényezője, vagy más néven szélessége.



9. ábra: PWM jel különböző kitöltési tényezővel

Az impulzusszélesség-modulált (Pulse Width Modulation, PWM) jelet nagyon gyakran használják különböző folyamatok vezérlésére, különösképp villanymotorok terén.

Semmi baj, ha még ezek alapján nem érted, hogy hogyan működik az ADC, DAC, PWM. Ezeket, és a továbbiakban bemutatott kommunikációs portokat is a későbbi fejezetekben részletesen bemutatjuk neked.

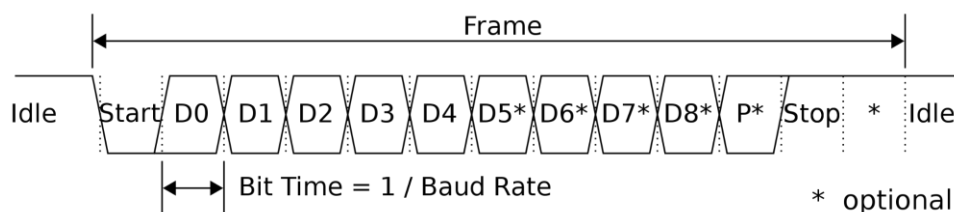
KOMMUNIKÁCIÓS PORTOK

A továbbiakban megemlítenék néhány kommunikációs lehetőséget a teljesség igénye nélkül.

Néhány láb speciális, kommunikációs funkciókat is képes ellátni, amennyiben megfelelően konfiguráljuk a perifériát.

UART

Az UART (Universal Asynchronous Receiver/Transmitter) az aszinkron kommunikáció kezelésért felelős egység, általában a vezérlőn belül kerül kialakításra. Az UART-hoz kapcsolódó lábak (a fejlesztőpanelen az UART_TX a PA2-es lábon az UART_RX pedig a PA3-as lábon található) többségében hardveres megszakításokhoz vannak kapcsolva, így egyszerűen értesíthető a processzor az új adat beérkezéséről. A kommunikáció egy start bittel kezdődik, ezt követik az adatbitek, az adatbitek után következik a paritásbit, végezetül a kommunikációt lezáró egy vagy több stopbit. Amennyiben a kommunikáció során bithiba keletkezik, akkor ezt a paritásbit segítségével szerencsés esetben detektálhatjuk.



10. ábra: UART kommunikációs üzenet

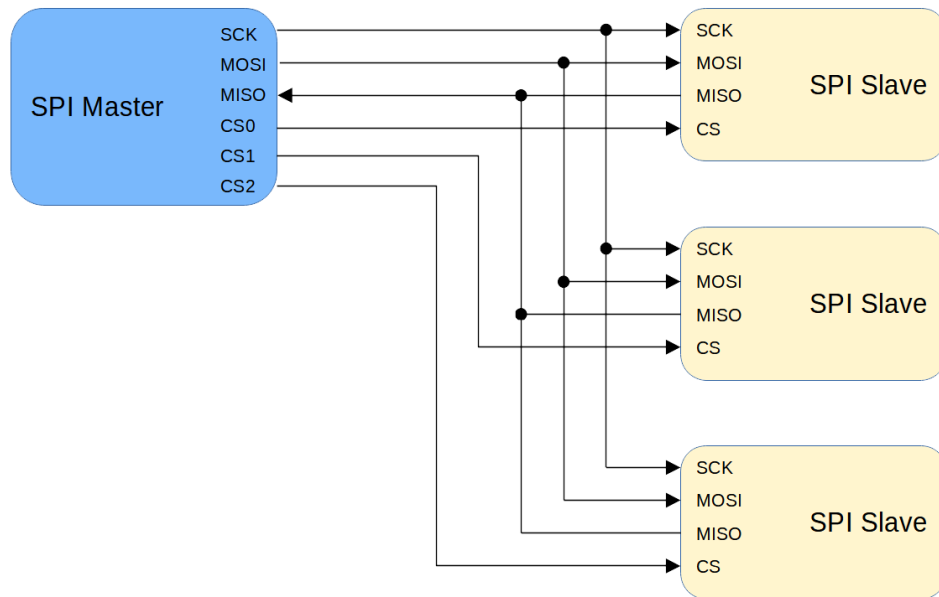
Forrás: AmenophisIII [CC0 1.0 (<https://creativecommons.org/publicdomain/zero/1.0/>)]
https://commons.wikimedia.org/wiki/File:UART_Frame.svg

SZINKRON SOROS KOMMUNIKÁCIÓ

A legtöbb mikrovezérlőnek van külön dedikált modulja a leggyakrabban használt szinkron soros kommunikációkra (SPI és I²C).

SPI

Az SPI (Serial-Peripheral Interface) kommunikációt leggyakrabban beágyazott rendszerekben belül, egymástól rövid távolságra (maximum 8 méter) elhelyezkedő eszközök között alkalmazzák, Master - Slave struktúra kialakításával. Ez egy alá-fölé rendelt kapcsolatot jelent, ahol az eszközöknek két szerepe lehet, ezek a Master és a Slave. Egy Masterhez több Slave is csatlakozhat, és a kommunikációban a Master határozza meg az adatáramlás idejét és irányát. Az SPI-on történő összes kommunikációt a Master vezérli. Külön vezetékek szolgálnak az adat küldésére és fogadására (MOSI és MISO), az órajelre (SCK) és minden egyes Slave-nek külön engedélyező jele van (SS#).



11. ábra: SPI busz felépítése

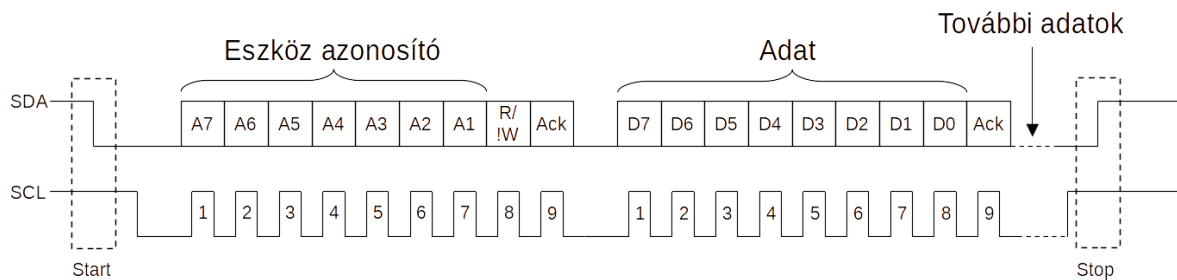
Ez a busz négy vonallal rendelkezik, a jeleket a master irányából nézzük.

- MOSI (Master Output Slave Input): Master által a slave-nek küldött adat kerül erre a vonalra
- MISO (Master Input Slave Output): Slave által a master-nek küldött adat kerül erre a vonalra
- SCK (Serial Clock): Órajel.
- SS (Slave Select, szokták még CS - Chip Select jelnek is nevezni): A Master ezzel a jellel választja ki, hogy melyik Slave eszközzel szeretne kommunikálni. Több Slave eszköz kezelésénél több Slave Select jelre is szükség van.

I2C

Az I2C (Inter-Integrated Circuit) kommunikációs módszer egy multi-master, multi-slave struktúra kialakítását teszi lehetővé. Jellemzően alacsony sebességű és kis távolságú, fedélzeti rendszerek között alakítják ki.

Itt is minden esetben a Master kezdeményezi az adatátvitelt, azonban itt csak 2 vezetékot használunk a kommunikációra. Az egyik a Master szolgáltatja az órajelet, a másikon pedig a ki- és bemenő adat áramlik. A Slave-eknek egyedi azonosítójuk van (Device ID), ezzel címezi meg a Master, hogy kinek szól az üzenet.



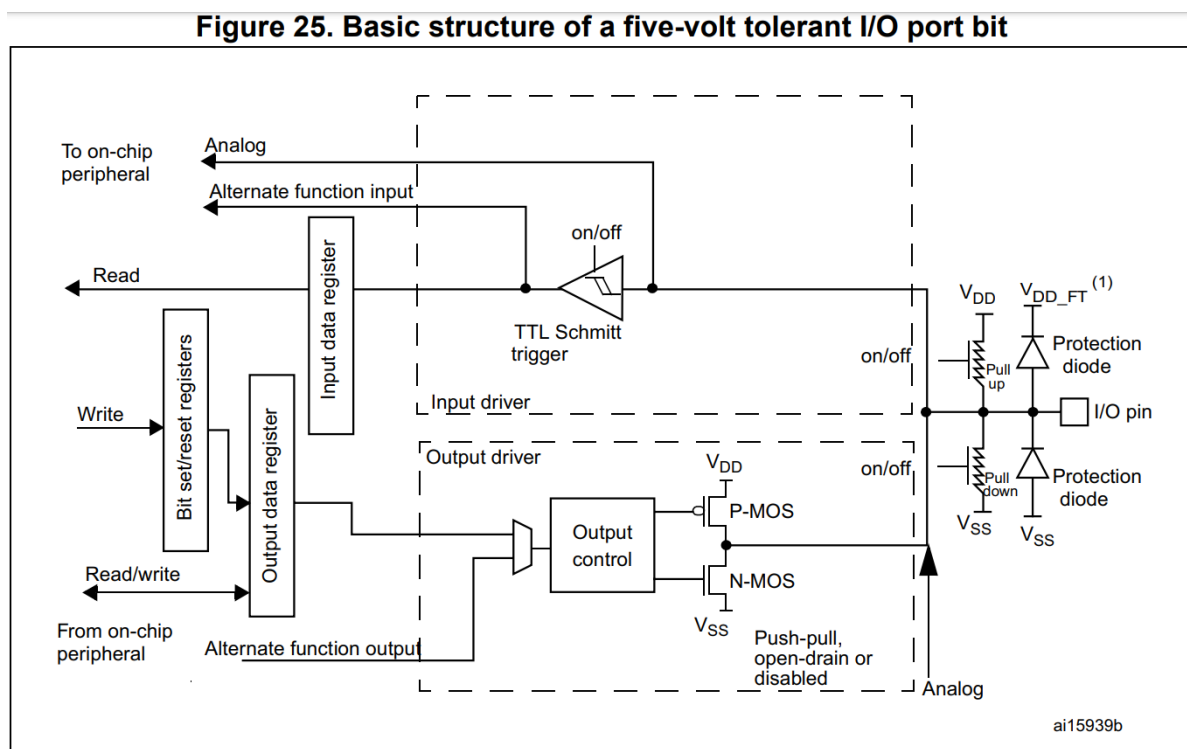
12. ábra: I2C kommunikáció, címzés és adattovábbítás

Ez a busz két fő vonallal rendelkezik:

- SDA (Serial Data): Adatvonal
- SCL (Serial Clock): Órajel

FELÉPÍTÉS

Most, hogy már tudjuk, milyen sokoldalúan használhatjuk fel ezeket a lábakat, nézzük meg, hogyan épülnek fel a mikrovezérlőn belül a lábakhoz tartozó áramkörreszletek.



13. ábra: STM32 mikrovezérlő I/O-portjának felépítése

Forrás: STMicroelectronics

https://www.st.com/resource/en/reference_manual/dm00031020-stm32f405-415-stm32f407-417-stm32f427-437-and-stm32f429-439-advanced-arm-based-32-bit-mcus-stmicroelectronics.pdf

A külvilág felől a vezérlő belseje felé haladva (a kép jobb szélétől indulva) közvetlenül az I/O pin után, a vezérlőn belül található a túlfeszültségvédő diódák (Protection diode), ezek akadályozzák meg,

hogy a lábra túl magas (pozitív) vagy túl alacsony, vagyis a földpotenciálhoz képest negatív feszültség kerüljön.

Ezután láthatjuk a ki- és bemeneti fokozatokat (szaggatott vonallal bekeretezett rész).

BEMENETI MÓD

Amennyiben a lábat bemenetként konfiguráljuk (Bemeneti mód - Input driver), úgy a kimeneti buffer (Output data register) letiltásra kerül. Eközben engedélyezőjelet kap a Schmitt-trigger.

Az elektronikában a Schmitt-trigger egy hiszterézissel rendelkező komparátor áramkör (lásd: Kristálytisza Elektronika első részének 14. tananyagrésze). Figyeli a bemenő feszültség szintet, és a kimenetét ennek függvényében változtatja. Ha a bemeneten a feszültség elért egy bizonyos szintet a kimenete magas logikai értékre vált. Amikor a bemeneten a feszültség egy megadott szint alá csökken, akkor a kimenet alacsony logikai értékre vált. A hiszterézis miatt a két kapcsolási küszöb nem esik egybe.

A Schmitt-trigger itt azokat a logikai feszültség szinteket határozza meg, ahonnan magas vagy alacsony logikai értékek veszi a vezérlő a bemenetet.

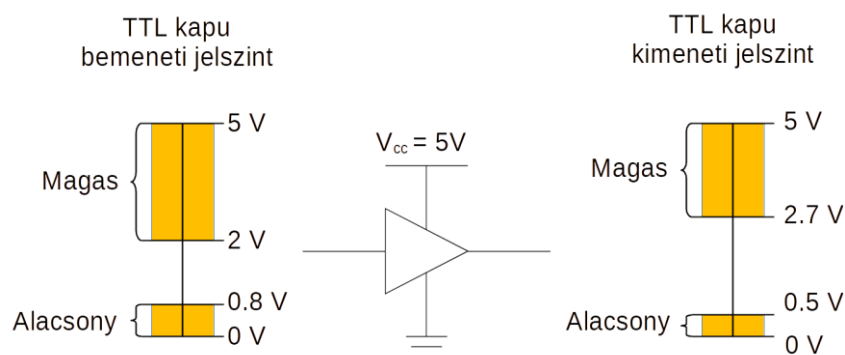
A logikai áramkörök, alacsony ("0") vagy magas ("1") logikai értéket érzékelnek a bemenetükön és szolgáltatnak a kimenetükön. Egy ideális világban az alacsony értékhez csak a 0 V feszültség szint, míg a magas szinthez a tápfeszültség értéke tartozna és ezektől sosem térnének el az eszközök.

A valóságban sajnos ritkán sikerül logikai jelekkel elérnünk ezeket a szélsőséges feszültség értékeket. Ez többek között annak is köszönhető, hogy az áramkörökben található tranzisztorok sem ideálisak, nem képesek teljes mértékben a tápra vagy földre kapcsolni.

A feszültség szintek szórása okozta probléma kiküszöbölésére alkották meg a ki- és bemeneti feszültségtartományok leképezését a logikai szintekre.

A bemeneti oldalon az alacsony ("0") szinthez a 0 - 0.8 V, magas ("1") szinthez a 2 - 5 V feszültség tartományt rendelték hozzá.

Kimenet esetén a 0 - 0,5 V tartomány jelenti az alacsony, a 2,7 - 5 V pedig a magas jelszintet.



14. ábra: A TTL logikában tolerált feszültség szintek

A Schmitt-trigger bemenetén két ellenállást láthatunk, amelyeken keresztül a bemenetet tápra, vagy földre tudjuk kapcsolni. Az ilyen ellenállásokat fel- illetve lehúzó ellenállásoknak hívjuk. Ezek attól függően kapcsolódnak a lábhoz, hogy milyen üzemmódot állítottunk be.

Pull-up üzemmód: A felső ellenállást a tápfeszültség és a láb közé kapcsolja (on), az alsó ellenállás nincs bekötve (off), ekkor a láb alapállapotban magas logikai értéket mutat, külső eszköz pedig alacsony logikai értékre képes lehúzni.

Pull-down üzemmód: Az alsó ellenállást a föld és a láb közé kapcsolja (on), a felső ellenállás nincs bekötve (off), ekkor a láb alapállapotban alacsony logikai értéket mutat, külső eszköz pedig magas logikai értékre képes felhúzni.

Nagyimpedanciás (Floating) üzemmód: Ekkor sem az alsó sem a felső ellenállás nem kapcsolódik a lábhoz. Alapállapotban a bemenet nem meghatározott értékű, azt mondjuk, hogy lebeg. Ekkor a külső eszköz képes kell legyen az alacsony és a magas logikai értéket is önállóan előállítani az elvárt működéshez.

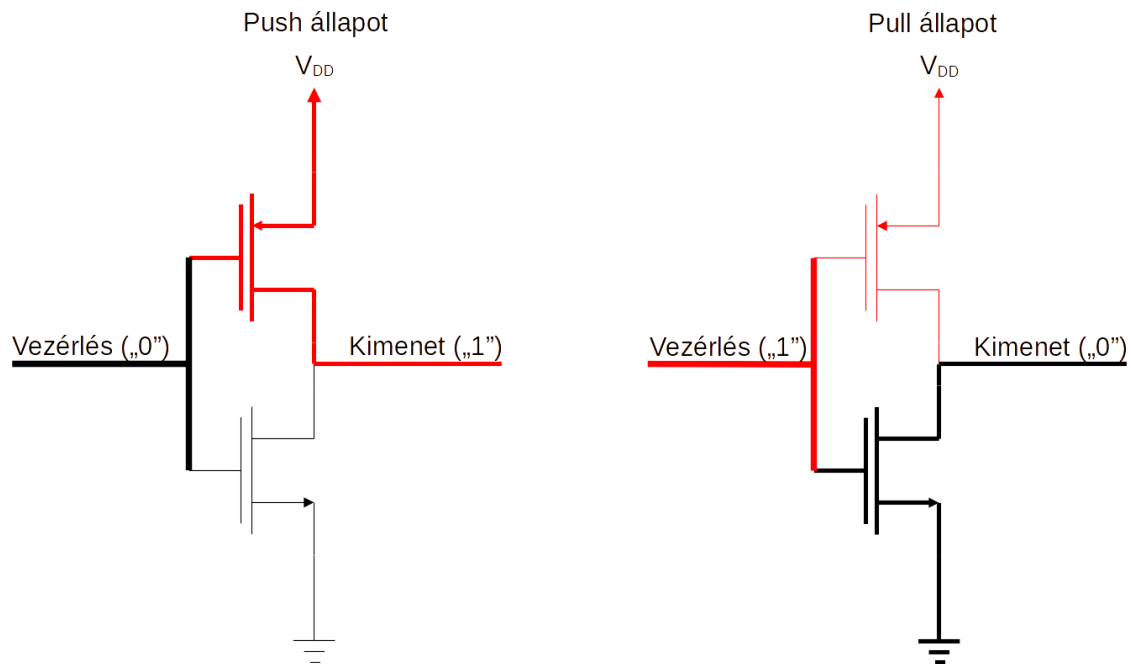
A fel- és lehúzóellenállások megakadályozzák a bemenetek lebegését, többek között az ebből adódó véletlen kapcsolásokat is.

KIMENETI MÓD

Amikor a lábat kimenetként konfiguráljuk (Kimeneti mód - Output drive), képes lesz alacsony vagy magas logikai értéket kiadni. Amennyiben az alkalmazás megindokolja, a fel- és lehúzó ellenállások itt is használhatók. A továbbiakban a kimeneti módokat ismertetem.

Push-Pull üzemmód

Általában ez az alapkonfiguráció, a GPIO láb ekkor képes forrásként (push) és nyelőként (pull) is működni. Mikor a kimeneten alacsony logikai értéket állítunk be, a láb egy FET-en keresztül a földpotenciálra, míg magas logikai érték esetén egy FET-en keresztül a tápfeszültségre kapcsolódik. Így a kimenetet aktívan tudjuk mind alacsony- mind magas logikai szinttel meghajtani.



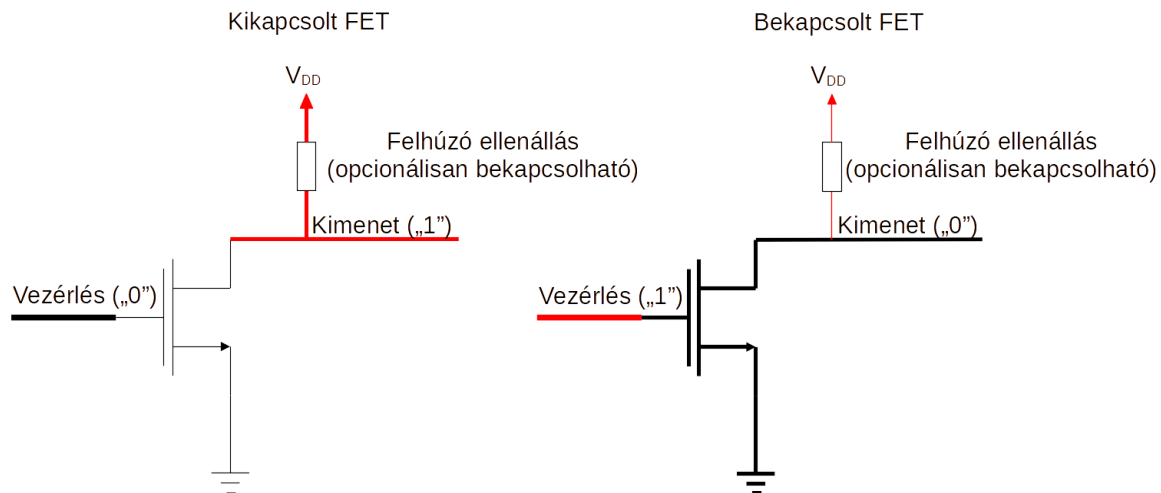
15. ábra: Push-Pull üzemmód szemléltetése FET-ekkel (Field Effect Transistor - Tervezérlésű tranzisztorokkal)
Open-drain / Open-collector üzemmód

A "collector" elnevezés itt egy BJT (Bipolar Junction Transistor - bipoláris tranzisztor) kollektorára, míg a drain egy FET (Field Effect Transistor - tervezérlésű tranzisztor) "drain"-jére vonatkozik.

Itt a kimenet csak nyelőként képes működni. Két vezérelt állapota van, a kimenet vagy földpotenciálra, vagy nagy impedanciás állapotba kerül. Külső felhúzóellenállással a kimeneten magas logikai érték is elérhető.

Ezen kialakítás előnye, hogy több eszköz összekötése esetén nem fordulhat elő összehajtás. Vagyis nem fordulhat elő olyan állapot, hogy egy adott eszköz logikai magas szintet kényszerít a vonalra, míg egy másik eszköz logikai alacsony szintet, amely nagy áramot eredményezne a két eszköz között, így azok meghibásodását okozva.

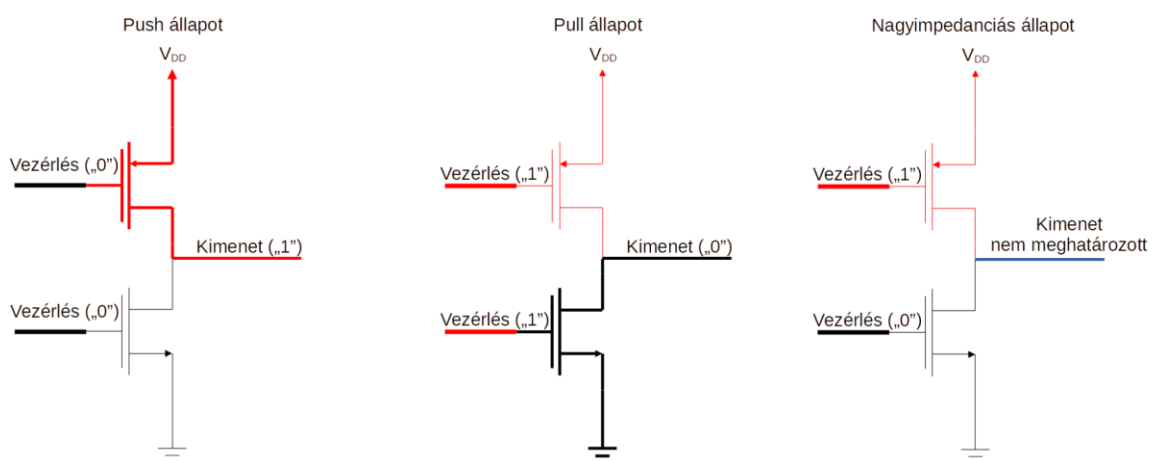
Ez az üzemmód például I²C buszon történő kommunikáció esetén játszik fontos szerepet, ahol a két vezeték (SDA, SCL) felhúzó ellenállásokkal látjuk el.



16. ábra: Open drain üzemmód szemléltetése FET-ekkel

Tri-State kimenet

Kombinálja az előző megoldásokat. A kimenet értéke lehet alacsony vagy magas logikai értékű és nagy impedanciás állapotba is vezérelhető. Leggyakoribb felhasználása a buszvezeték meghajtása. A buszvezetékre csatlakoztatott tri-state kimenetű áramkörök közül mindig csak egyet szabad engedélyezni, a többi kimenete "lebeg", így nem befolyásolják a buszvezeték állapotát, és nem is károsítják egymást.



17. ábra: Tri-State kimenet szemléltetése FET-ekkel

KIMENETEK KAPCSOLÁSA

A LED villogtatás, a beágyazott rendszerek “Hello World”-je. Ezzel ugyanis könnyedén tesztelhetjük mind a kapcsolás néhány elemét, mind pedig a program futását.

Már ismerjük, hogy fizikailag hogyan működnek a lábak. Most nézzük meg milyen parancsokkal lehet a különböző konfigurációkat beállítani!

A szükséges példakódot már elkészítettük, neked csupán megértened és felhasználnod szükséges. A lépések megértésével később magad is megvalósíthatod az egyes funkciókat.

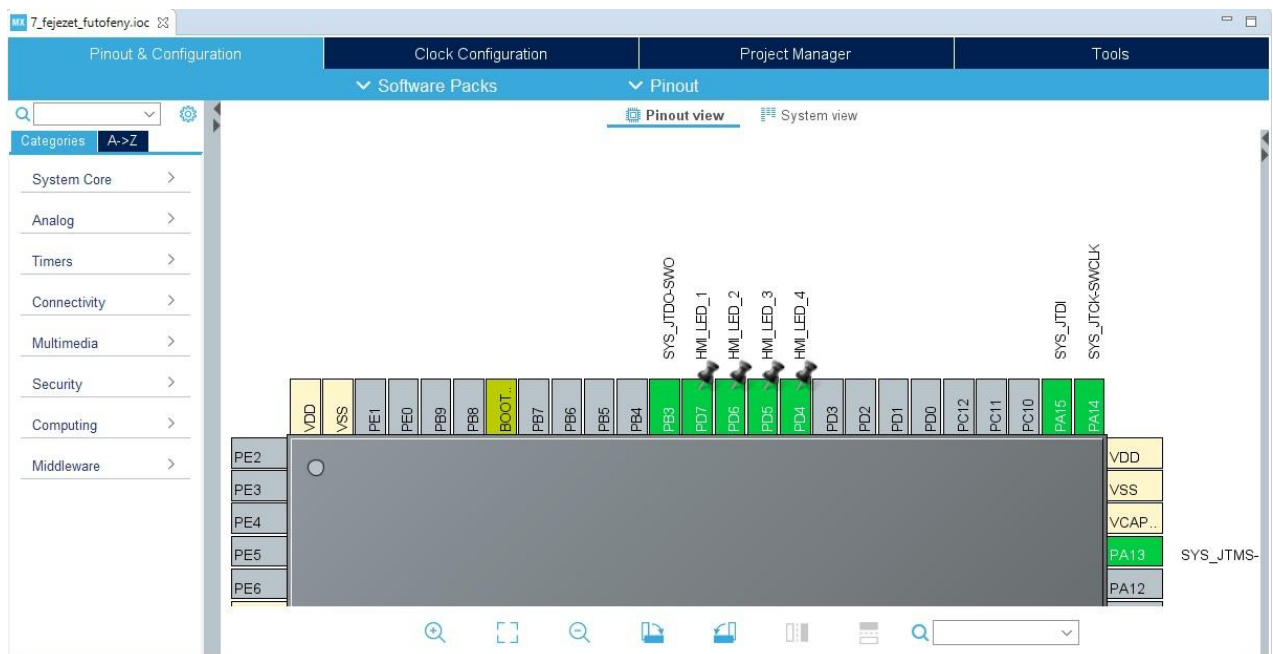
Első lépésként nyisd meg a fejezethez tartozó **7_fejezet_futofeny** nevű projektet.

Ha egyszer szükséged lesz a lábak megfelelő beállítására, azt legegyszerűbben a fejlesztői környezet hardver-konfigurátorban teheted meg.

Ehhez nyisd meg a projekten belül található **7_fejezet_futofeny.ioc** fájlt, ekkor megjelenik a mikrovezérlő sematikus ábrája és a lábkiosztás.

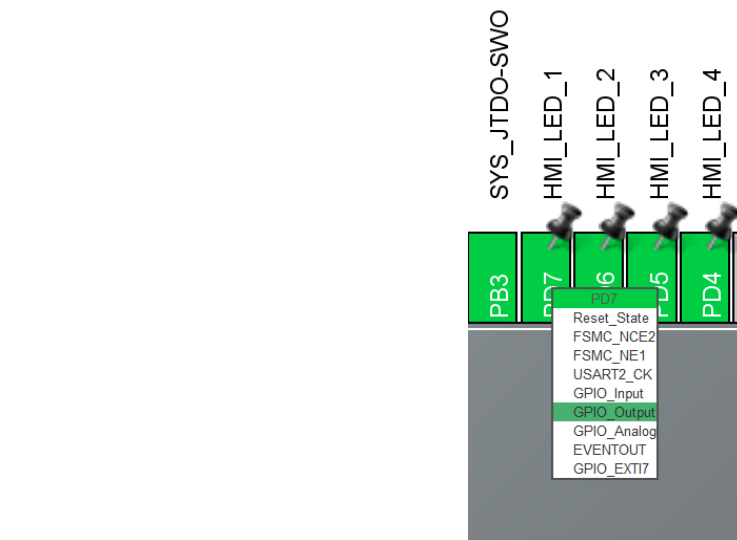
Összesen 4 db visszajelző LED található a panelen (LED1-LED4 felirattal vannak ellátva), ezek a vezérlőn az alábbi lábakhoz kapcsolódnak:

- HMI_LED_1 a PD7-es lábhoz
- HMI_LED_2 a PD6-os lábhoz
- HMI_LED_3 a PD5-ös lábhoz
- HMI_LED_4 a PD4-es lábhoz



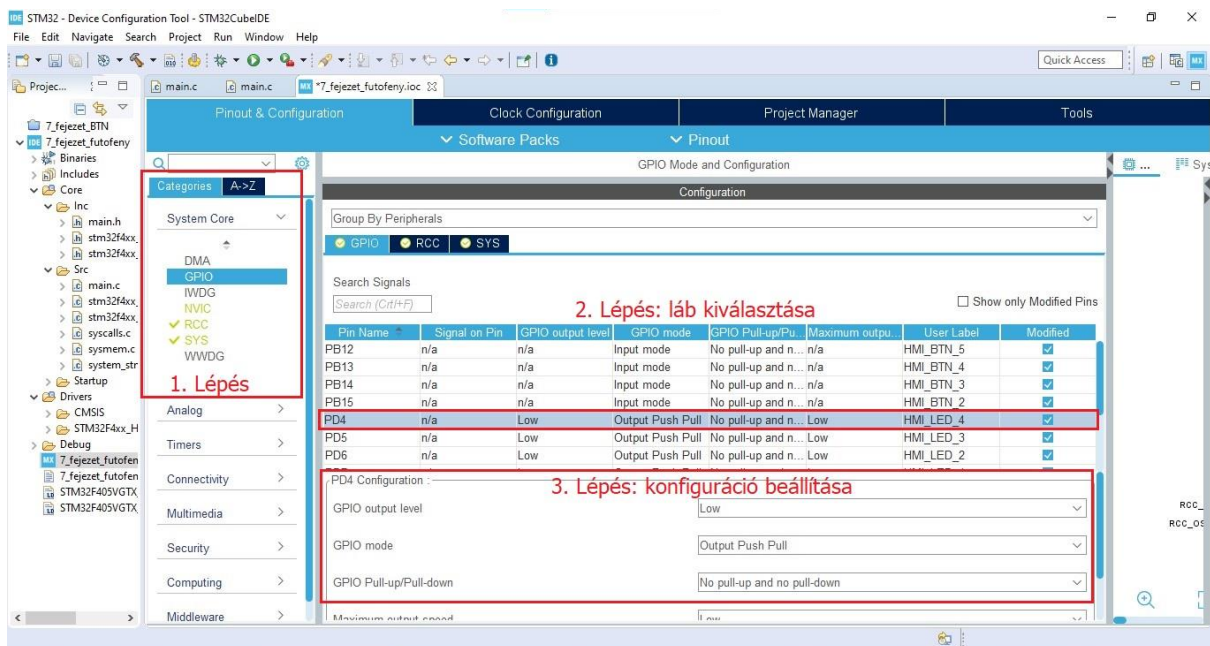
18. ábra: Visszajelző LED-ek a Hardver konfigurátorban

A konfigurálni kívánt lábra kattintva megjelenik egy menü, amiből kiválasztható, miként szeretnénk beállítani a lábat, mi a jelen alkalmazáshoz GPIO_Output módra állítottuk.



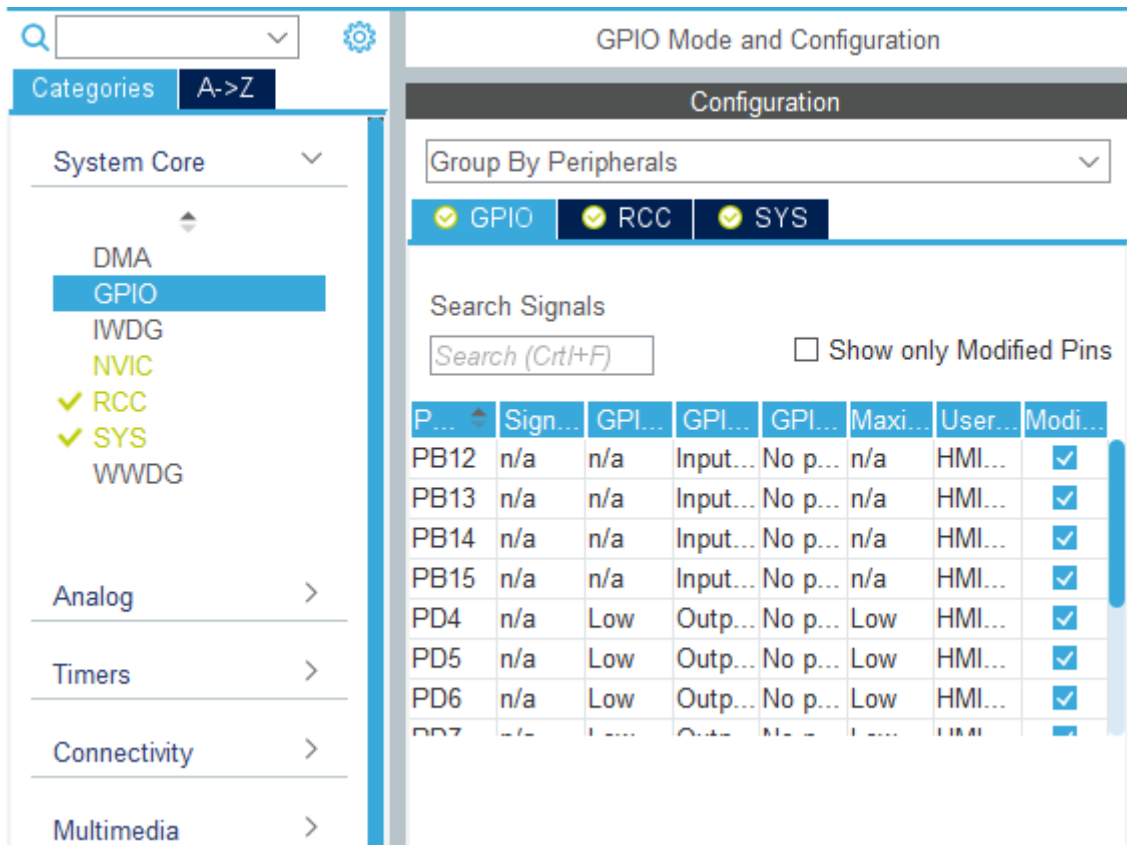
19. ábra: Láb beállítási lehetőségek

Miután minden lábat az igényeknek megfelelően beállítottunk, az oldalsó kategóriák fül SystemCore/GPIO menüpontját választva a használatra kiválasztott lábakhoz tartozó részletes beállítások jelennek meg. Az adott lábhoz tartozó sorra kattintva lehetőségünk van legördülő menüből kiválasztani a megfelelő beállítást.



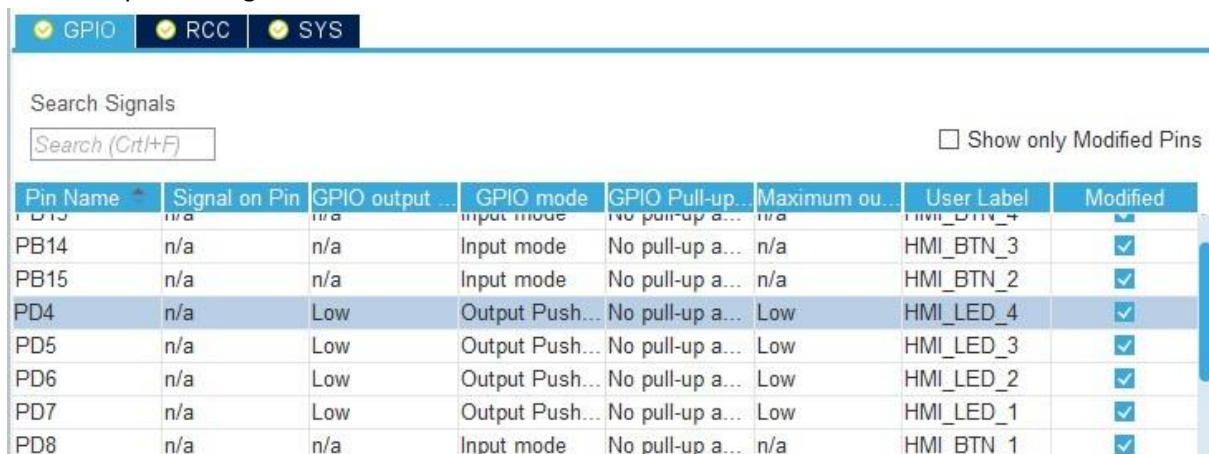
20. ábra: GPIO konfiguráció lépései

1. lépés: Válasszuk ki a System Core, GPIO menüpontot



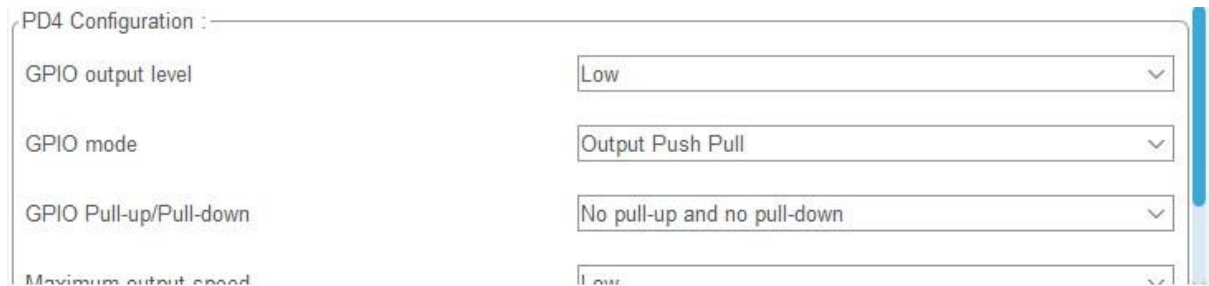
21. ábra: GPIO menüpont felépítése

2. lépés: Konfigurálni kívánt láb kiválasztása.



22. ábra: Különböző lábak konfigurációja

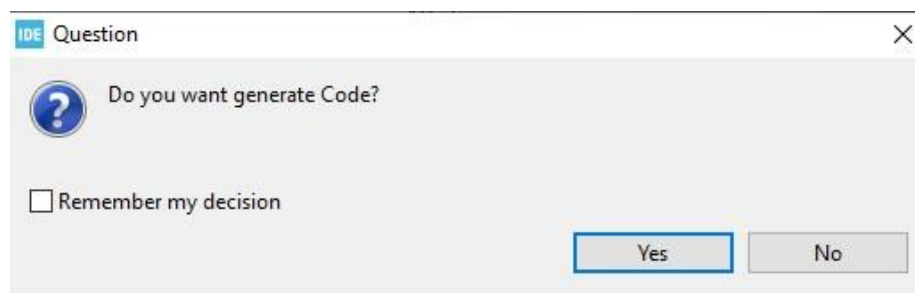
3. lépés: Konfiguráció beállítása, a felhasználási módnak megfelelően.



23. ábra: Adott láb paramétereinek beállítása

A képen látható, hogy a kimenet alapértelmezetten alacsony logikai szintű lesz, az üzemmódja Push-Pull és nem használunk sem felhúzó, sem lehúzó ellenállást,

A példakód tartalmaz minden beállítást, amennyiben te is állítanál valamin és rákattintasz a mentés gombra, egy párbeszédpanel jelenik meg. Ne feledd, ez a képernyő csak akkor jelenik meg, ha módosítasz a beállításokon!



24. ábra: Kódgenerálás üzenet

A Yes gombra kattintva a fejlesztőkörnyezet előállítja és a megfelelő helyre illeszti a kódot, így nem nekünk kell azt begépelni (azonban, ha szeretnénk semmi sem tiltja :)). Amennyiben a beágyazott kód futása közben szeretnénk átkonfigurálni a beállításokat (például az ADC-t másik porthoz kapcsolni, mint amit a konfigurátorban megadtunk), azt begépelte kódrészlettel tehetjük meg.

A konfigurátorban elvégzett GPIO beállításokból generált kód az **MX_GPIO_Init()** függvénybe kerül. A függvényt a **main.c** fájlban találod. Ebben először a GPIO struktúra létrehozása történik, majd az alábbi kódsorokkal az adott portok órajelének engedélyezése.

```
/* GPIO Ports Clock Enable */
__HAL_RCC_GPIOH_CLK_ENABLE(); // H GPIO port órajel engedélyezés
__HAL_RCC_GPIOB_CLK_ENABLE(); // B GPIO port órajel engedélyezés
__HAL_RCC_GPIOD_CLK_ENABLE(); // D GPIO port órajel engedélyezés
__HAL_RCC_GPIOA_CLK_ENABLE(); // A GPIO port órajel engedélyezés
```

Pinek beállítása kezdeti állapotba:

```
/* GPIO pin kimenet beállítás */  
HAL_GPIO_WritePin(GPIOD, HMI_LED_4_Pin|HMI_LED_3_Pin|HMI_LED_2_Pin|HMI_LED_1_Pin,  
GPIO_PIN_RESET);
```

Majd ezután történik az adott portok és hozzájuk kapcsolódó lábak beállítása.

Itt kerülnek definiálásra a konfigurátorban beállított értékek, úgy, mint ki- és bemenet, fel- és lehúzó ellenállás stb.

```
/*HMI_BTN_5_Pin, HMI_BTN_4_Pin, HMI_BTN_3_Pin, HMI_BTN_2_Pin-ek konfigurálása */  
GPIO_InitStruct.Pin = HMI_BTN_5_Pin|HMI_BTN_4_Pin|HMI_BTN_3_Pin|HMI_BTN_2_Pin;  
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;           // Mód beállítás  
GPIO_InitStruct.Pull = GPIO_NOPULL;               // Fel/lehúzó ellenállás  
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);          // Inicializálás  
  
/*HMI_BTN_1_Pin, HMI_SW_4_Pin, HMI_SW_3_Pin, HMI_SW_2_Pin, HMI_SW_1_Pin-ek  
konfigurálása */  
GPIO_InitStruct.Pin = HMI_BTN_1_Pin|HMI_SW_4_Pin|HMI_SW_3_Pin|HMI_SW_2_Pin|  
HMI_SW_1_Pin;  
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;           // Mód beállítás  
GPIO_InitStruct.Pull = GPIO_NOPULL;               // Fel/lehúzó ellenállás  
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);          // Inicializálás  
  
/*HMI_LED_4_Pin, HMI_LED_3_Pin, HMI_LED_2_Pin, HMI_LED_1_Pin-ek konfigurálása */  
GPIO_InitStruct.Pin = HMI_LED_4_Pin|HMI_LED_3_Pin|HMI_LED_2_Pin|HMI_LED_1_Pin;  
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;       // Mód beállítás  
GPIO_InitStruct.Pull = GPIO_NOPULL;               // Fel/lehúzó ellenállás  
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;      // Sebesség beállítás  
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);          // Inicializálás
```

Ezután már csak annyi dolgunk van, hogy szoftverből ki- és bekapcsoljuk a LED-eket.

Az alábbi példa egy futófény megvalósításán keresztül mutatja be a szükséges programozástechnikai részleteket.

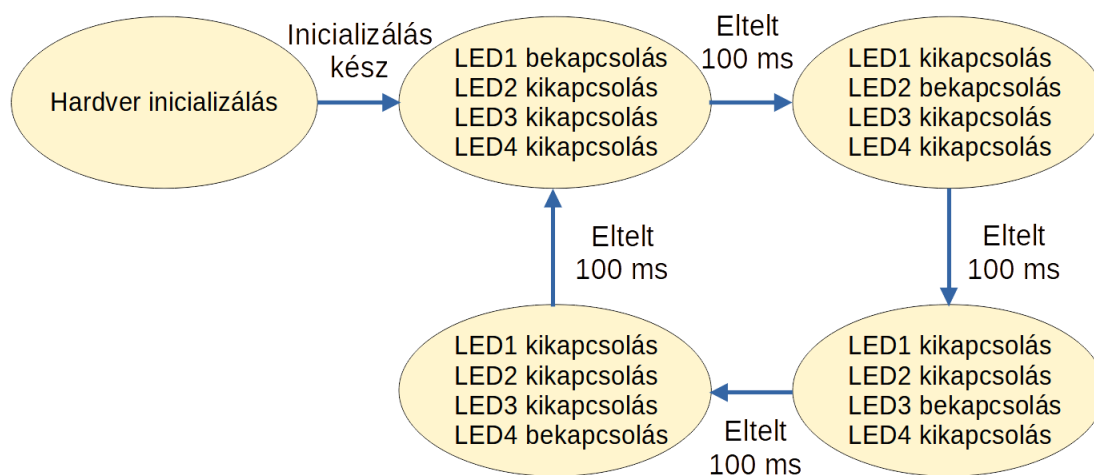
HAL_GPIO_WritePin() függvénnyel tudjuk az adott kimenetet (kimenetként konfigurált GPIO lábat) alacsony vagy magas logika értékre állítani. Erről a függvényről az 5. tananyagrészen részletesen olvashatsz.

A futófényt megvalósíthatjuk állapotgéppel is.

Kitekintés: Állapotgép

Az állapotgép egy viselkedést leíró modell, melyben definiálhatunk állapotokat (rendszer működésének leírása adott körülmények között) és az állapotok közötti állapotátmeneteket (adott információ hatására, adott állapotból mi legyen a következő állapot). A legszemléletesebb reprezentációja az állapotgépes leírásnak az állapotdiagram, amely az állapotokat és az átmeneteket tartalmazza.

Ebben a leírásban az állapotokat ellipszisek jelölik (az állapotban valamit csinál a rendszerünk), az állapotok közötti átmeneteket nyilak jelölik (a nyilakra írjuk azt a feltételt, aminek teljesülése esetén az átmenet bekövetkezik, például eltelt 100 ms az adott állapotban). Számunkra ez most egy tökéletes megoldás lenne, ugyanis a state változó tartalmazhatja az aktuális állapotot (mely LED-ek világítanak és melyek ne). A futófény működéséből adódóan minden LED csak egy állapotban lehet bekapcsolva, a többiben nem szabad világítania. A LED bekapcsolása után várunk, majd a várakozási idő letelte után (állapotátmenet) léptetjük az állapotgépet.



25. ábra: Állapotgép felépítése

Kitekintés: Ternary operátor

Az állapotgépet legegyszerűbben switch-case szerkezettel tudjuk megvalósítani, amivel az előző tananyagrészen már találkozhattál. Most mégis egy másik, szintén megfelelő programozástechnikai megoldást szeretnék mutatni neked. Hasonlót, mint amit az előző tananyag rész összefoglalójának a kitekintés részében láthattál.

Először ismerkedjünk meg a (? :), másnéven ternary operátorral. A C nyelvben ez az egyetlen 3 operandusú operátor, azaz 3 érték alapján végez műveletet. Működése leginkább az if-else szerkezetre hasonlít.

Használata a következő szerint történik: $A ? B : C$, ahol a művelet végrehajtása az A operandus kiértékelésével kezdődik. Amennyiben az A kifejezés igaz, úgy a művelet eredménye B értéke lesz, ha A értéke hamis, akkor pedig C.

Ha most még nem egészen érted, hogyan is működik, semmi gond, a következő példával már

minden világos lesz.

Gondoljunk arra, hogy hőmérsékletet mérünk (azt úgyis gyakran szoktunk :)). Van egy fűtési rendszerünk, amit 22 °C alatt szeretnénk bekapcsolni, felette pedig kikapcsolni.

Legyenek a változóink Fűtés és Hőmérséklet. A kikapcsoláshoz tartozzon a 0 érték, a bekapcsoláshoz pedig az 1 érték.

Ezt a programban úgy tudjuk az A ? B : C operátorral leírni, hogy

Fűtés = (Hőmérséklet < 22) ? 1 : 0;

Mint mondtam, logikailag az if - else szerkezetre hasonlít, ezért nézzük meg úgy is a fenti példát.

if (Hőmérséklet < 22)

{

Fűtés = 1;

}

else

{

Fűtés = 0;

}

Fontos megjegyezni, hogy a fenti egyszerű példában mind a két megoldás ugyanazt az eredményt adja, a háttérben azonban más dolgok történnek. A (? :) operátort nyugodtan alkalmazzuk feltételes értékadásra, az if-else szerkezetet pedig bonyolultabb logikák megvalósításához.

A LED-ek kapcsolása között a HAL_Delay() függvénnyel várakozunk 100 milliszekundumot. Várakozás nélkül a vezérlő olyan gyorsan kapcsolná ki és be a LED-eket, hogy fel sem tűnne a szemünknek a változás.

Az állapotgépes megvalósításhoz szükségünk lesz az állapotok tárolására, erre egy state nevű változót fogunk használni.

A state változó értékétől függően egy adott LED világít, vagy nem világít. Az állapotváltozó különböző értékei különböző állapotokat jelentenek. Legyenek ezek az állapotok az 1,2,4,8. Az & operátorral az adott változók bitjein logikai ÉS műveletet végzünk, ezt konyhanyelven "éselésnek" nevezzük. A 0x01, 0x02, 0x04, 0x08 számokban közös, hogy bináris alakjuk 1 db 1-et tartalmaz. Az éselés eredménye is egy szám lesz. Tehát ha a state változó értéke a 0x0D (binárisan felírva 00001101) szám lenne, akkor a következő eredmény adódna:

- state & 0x01 (= 0x01), a kiértékelés igaz,
- state & 0x02 (= 0x00), a kiértékelés hamis,
- state & 0x04 (= 0x04), a kiértékelés igaz,
- state & 0x08 (= 0x08), a kiértékelés igaz.

Láthatjuk, hogy a 0 szám logikai értéke hamis, de minden más számhoz logikai igaz érték tartozik.

```
while (1)
{
    //state változó értékvizsgálat
    HAL_GPIO_WritePin(HMI_LED_1_GPIO_Port, HMI_LED_1_Pin, state & 0x01 ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(HMI_LED_2_GPIO_Port, HMI_LED_2_Pin, state & 0x02 ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(HMI_LED_3_GPIO_Port, HMI_LED_3_Pin, state & 0x04 ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(HMI_LED_4_GPIO_Port, HMI_LED_4_Pin, state & 0x08 ?
GPIO_PIN_SET : GPIO_PIN_RESET);

    state <<= 1;          //balra shiftelés (2-vel szorzás)

    //state változó értékének visszaállítása, ha elérte a 16-ot
    if (state == 0x10)
    {
        state = 0x01;
    }

    HAL_Delay(100);       //ciklus várakozási idő

    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
```

Az értékvizsgálat után az állapotváltozót 1-gyel balra shifteljük (azaz 2-vel szorozzuk). Ha a változó elérte a 0x10 hexadecimális értéket (decimálisan 16), a változó értékét 1-re állítjuk. Beláthatjuk, hogy a state változó értéke a következő sorozat: 1, 2, 4, 8, 1, 2...



BEMENET BEOLVASÁSA

A legegyszerűbbnek gondolt bemeneti eszköz a nyomógomb, de most elmondom, miért is nem olyan egyszerű :). Amikor a mikrovezérlővel egy gomb lenyomását szeretnénk vizsgálni, előfordul, hogy nem várt dolgok történhetnek, például egy lenyomásra több lenyomást is érzékel a vezérlőnk.

Jogos a kérdés: „*hogyan van ez?*”

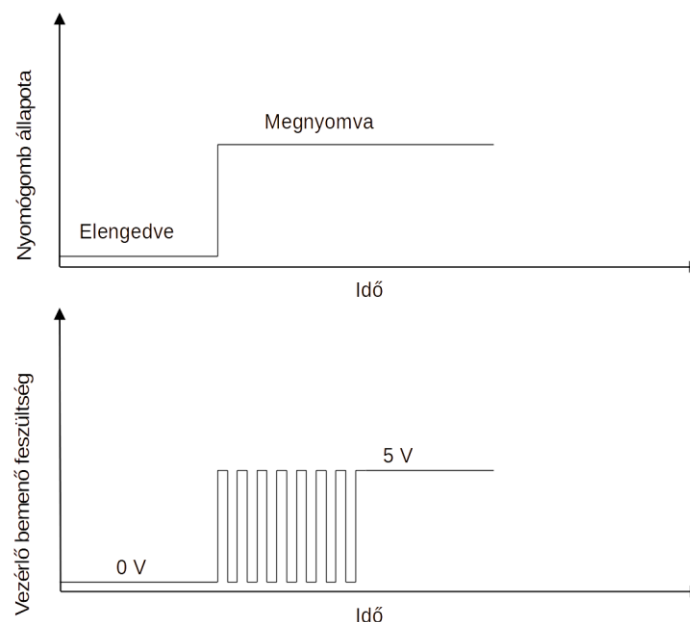
Mi csak egyszer nyomjuk meg a gombot, akkor mi történik?

Kezdjük az elején:

Tételezzük fel, hogy van egy ideális nyomógombunk (a valóságban sajnos ilyen nincs). Az ideális NO (normally open - alapállapotban nyitott) nyomógomb lenyomásakor a kontaktusait rögtön zárja, felengedéskor pedig rögtön nyitja. A valóságban kapható kapcsolók és nyomógombok mechanikus alkatrészekből épülnek fel, rugalmas fém alkatrészeket tartalmaznak. Éppen ezért a kapcsoláskor az érintkezőknél rezgés keletkezik. A kapcsoló az átkapcsolást követően a nyitott és zárt állapot között oszcillál, akár néhány ezredmásodpercig (ms) is, ezt hívjuk pergésnek.

Mi ezt a mindennapi életben nem vesszük észre, de a vezérlő néhány ms alatt több ezer műveletet is elvégez. A mérnöki gyakorlatban ez például akkor okoz gondot, ha a mikrovezérlővel fel- (0→1 átmenet) vagy lefutó él (1→0 átmenet) vizsgálatát szeretnénk elvégezni. Beláthatjuk, hogy egy gombnyomásra ilyenkor több él fog keletkezni.

Az alábbi ábrán egy gombnyomás hatására keletkező felfutó él detektálása látható.

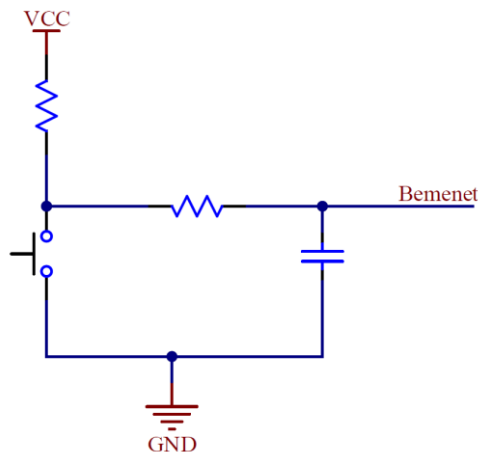


26. ábra: Nyomógomb figyelése a mikrovezérlővel

A pergés okozta zavart célszerű hardveres vagy szoftveres pergésmentesítő módszer alkalmazásával elkerülni.

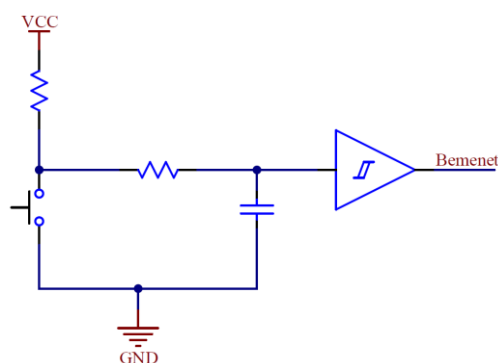
HARDVERES PERGÉSMENTESÍTÉS:

Ha a nyomógombhoz az alábbi ábra szerint hozzákötünk egy ellenállást (R2) és egy kondenzátort (C1), akkor az elnyeli a pergés során keletkező impulzusokat.



27. ábra: Pergésmentesítés RC szűrővel

Egy igazán szép megoldás RC tag kiegészítése Schmitt-triggerrel. Az általunk használt STM32 mikrovezérlő tartalmaz beépített Schmitt-triggert, mint ahogyan azt a “Bemeneti mód” részben megismerhettük. Amennyiben egy vezérlő nem tartalmaz Schmitt-triggert, úgy külső diszkrét alkatrészként érdemes a kapcsolásba beépíteni.



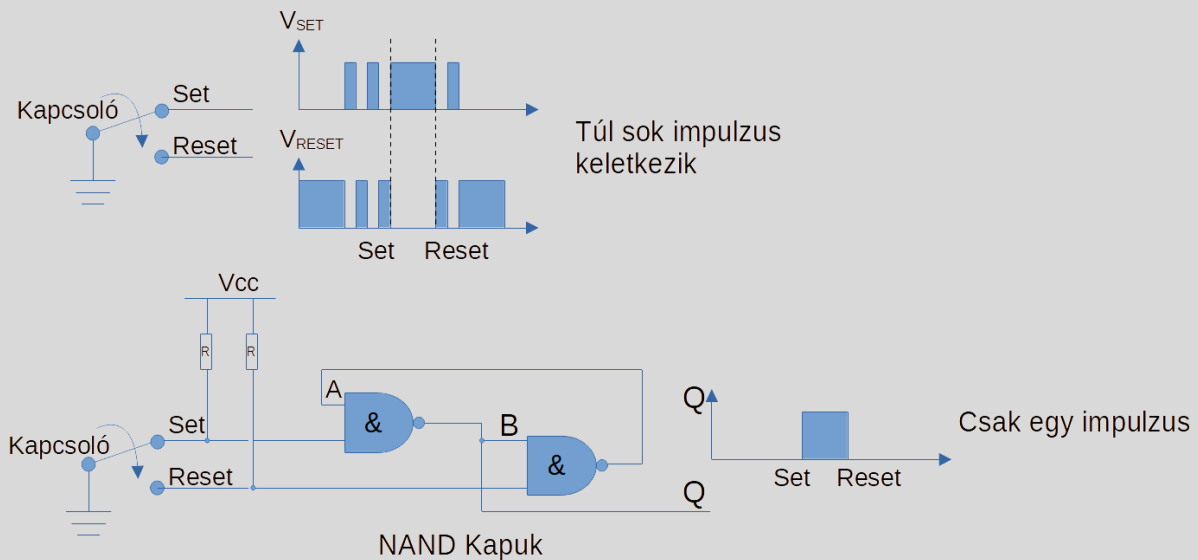
28. ábra: Pergésmentesítés RC szűrő és Schmitt-trigger segítségével

A szűrt jelet egy Schmitt-triggerrel figyeljük. A kapcsoló működtetésével a kondenzátort töltjük az ellenálláson keresztül. A gyors feszültségváltozásokkal (nagy frekvencia) szemben a kondenzátor rövidzárként viselkedik a föld felé (elvezeti / elnyomja azokat), míg a lassú változásokkal (alacsony frekvencia) szemben szakadásnak tekinthető, átengedi azokat az RC tag kimenetére. A Schmitt-triggernek az az érdekessége, hogy egy alacsonyabb feszültségszintnél billen le, és egy nagyobb feszültségszinten billen fel, tehát úgy működik, mint egy hiszterézises komparátor, tehát nem egy adott feszültségértékhez komparál, hanem egy feszültségtartomány alsó és felső határértékéhez (lásd: Kristálytisza Elektronika 14. tananyagrésze).

Az RC tagot úgy kell méretezni, hogy a kapcsoló pergése ne tudjon nagyobb ugrásokat kelteni, mint a Schmitt-trigger hiszterézise, ez biztosítja a pergésmentességet.

Kitekintés:

Egy másik, szintén tökéletesen működő megoldás egy SR flip-flop használata kétállású nyomógombok esetén.



29. ábra: Pergésmentesítés SR "flip-flop"-pal

Alapesetben balról az első kapu Set lábát a kapcsoló 0-ba húzza, így a NAND kapu kimenetén (lásd: Kristálytisza Elektronika 1. tananyagrésze) 1-es logikai érték lesz. A második kapu egyik bemenetét az első kapu kimenete hajtja meg (B jelzés az ábrán), a másik láb egy ellenállással szintén fel van húzva 1-es értékre, vagyis a kapu kimenete 0 lesz. Ezt megkapja az első kapu másik bemenete (A jelzés az ábrán). Ez a rendszer így stabil.

Ha elkezdjük a kapcsolót átváltani, a nyelv útközben egyik érintkezőhöz sem csatlakozik, így az első kapu egyik bemenete 1-be kerül (ellenálláson keresztül a tápra csatlakozik), de a másik (A jelű) bemenete továbbra is 0-t kap a második kaputól, tehát a tároló állapota nem változik. Amikor a nyelv hozzáér az alsó érintkezőhöz, a második kapu bemenete 0-t kap, a kimenete 1-be vált, az első kapu mindkét bemenete 1 lesz, a kimenete átmegy 0-ba, vagyis átbillen a tároló, és megint stabil állapotba kerül.

Amennyiben sok bemenetet kell kezelünk, úgy az alkatrészek már nagyon bonyolulttá teszik a kapcsolást és sok helyet is foglalnak. Ilyenkor érdekesebb egy másik utat, a szoftverből történő pergésmentesítést választanunk. Fontos megemlítenünk azonban, hogy sok bemenet független kezelése esetén a szoftveres pergésmentesítés is erőforrás-igényes. Ezért mindig végig kell gondolnunk, mi a legcélravezetőbb megoldás. Ha sok kapcsolónk van és viszonylag ritkán kapcsolgatjuk őket, akkor érdekesebb a szoftveres utat választanunk. Amennyiben a kapcsolókat gyakran kapcsoljuk, úgy célszerűbb a hardveres megoldás kialakítása.

SZOFTVERES PERGÉSMENTESÍTÉS

Az alábbi példában egy lehetőséget mutatunk a szoftverből történő pergésmentesítésre. A szoftveres pergésmentesítés lényege, hogy fizikailag a pergés bekövetkezik, mi azonban szándékosan csak egyszer vesszük figyelembe az átkapcsolást a vélt kapcsolási idő alatt.

Az előző példát változtassuk meg és egészítsük ki úgy, hogy a LED-ekkel egy négybites bináris számlálót valósítunk meg.

Már elkészítettük a megfelelő módosításokat, neked továbbra sincs más dolgod, mint megnyitni a fejezethez tartozó **7_fejezet_BTN** projektet, azon belül pedig a **main.c** fájlt.

Elsőként választunk egy küszöbértéket (általunk gondolt, vagy kitapasztalt maximális érték), amely számú periódus után nem következik be több pergés (*DEBOUNCING_TRESHOLD*).

Szükséges két változó, amiben a nyomógomb állapotát tároljuk. Az egyikben a gomb előző (*previous_state_debounce*), másikban pedig az aktuális állapotot (*current_state_debounce*) tároljuk.

El kell tároljuk az egymást követő periódusokban a korábbival megegyező állapotok számát (*count_debounce*) továbbá a le- és felfutó élek közötti átmenetet (*previous_state_edge*) is detektálni kell.

```
/* USER CODE BEGIN 2 */

#define DEBOUNCING_TRESHOLD 20           // pergésmentesítési küszöbérték

uint8_t count_debounce = 0;             // korábbival megegyező állapotok száma
uint8_t previous_state_debounce;        // gomb korábbi állapot
uint8_t current_state_debounce =
HAL_GPIO_ReadPin(HMI_BTN_1_GPIO_Port,HMI_BTN_1_Pin); // gomb aktuális állapot
uint8_t previous_state_edge = HAL_GPIO_ReadPin(HMI_BTN_1_GPIO_Port,HMI_BTN_1_Pin);
// éldetektálásra szolgáló változó

uint8_t counter = 0;                    //ciklusváltozó

/* USER CODE END 2 */
```

Vizsgáljuk a gomb állapotát és várunk, míg stabil állapotba nem kerül (nem változik az értéke). Fontos vizsgálnunk azt is, hogy a stabilnak vett állapotok (most 20) egymás után következtek-e. Amennyiben a jelenlegi és az előző állapot értéke megegyezik, növeljük a számlálót, ha pedig eltér, nullázzuk azt. Ha a számláló eléri a kitapasztalt maximális értéket, biztosak lehetünk benne, hogy a pergés lezajlott, a gomb stabil állapotba került.

Ha nem vizsgálnánk, vagy ha a stabilnak vett állapotok darabszámát kicsinek választanánk meg, akkor néha hibás működést tapasztalnánk. Előfordulhat olyan eset, hogy a stabil állapotok közötti pergéseket kapcsolásnak vennénk.

```

count_debounce = 0; // változó nullázás

//gomb állapotának vizsgálata, amíg elég stabil állapot nem volt
while (count_debounce < DEBOUNCING_TRESHOLD)
{
    previous_state_debounce = current_state_debounce;
    current_state_debounce = HAL_GPIO_ReadPin(HMI_BTN_1_GPIO_Port, HMI_BTN_1_Pin);

    //számláló értékének növelése, ha az előző állapot megegyezik a jelenlegivel
    if (previous_state_debounce == current_state_debounce)
    {
        count_debounce++;
    }
}

```

Miután már biztosan stabil állapotban van a kapcsoló, megvizsgáljuk, hogy történt-e kapcsolás (éldetektálás), amennyiben igen, úgy léptetjük a számlálót.

```

//éldetektálás, amennyiben történt élváltás, akkor ciklusváltozó növelés
if (current_state_debounce && !previous_state_edge)
{
    //LED vezérlő lábak beállítása a counter változó alapján
    HAL_GPIO_WritePin(HMI_LED_1_GPIO_Port, HMI_LED_1_Pin, counter & 0x08 ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(HMI_LED_2_GPIO_Port, HMI_LED_2_Pin, counter & 0x04 ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(HMI_LED_3_GPIO_Port, HMI_LED_3_Pin, counter & 0x02 ?
GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(HMI_LED_4_GPIO_Port, HMI_LED_4_Pin, counter & 0x01 ?
GPIO_PIN_SET : GPIO_PIN_RESET);

    counter++; //változó léptetés
}
previous_state_edge = current_state_debounce; //aktuális állapot elmentés

```

ANALÓG ÉS DIGITÁLIS VILÁG KAPCSOLATA

Itt csak egy rövid ismertetőt szeretnék nyújtani az analóg-digitális és a digitális-analóg átalakítók működéséről. Ezen eszközök részletes leírása a 11. és a 12. tananyagrészen megtalálható.

ANALÓG-DIGITÁLIS ÁTALAKÍTÓ

A környezetünkből érkező jeleket szeretnénk valahogy feldolgozni a vezérlőegységgel. Ha arra vagyunk kíváncsiak, hogy egy kapcsolót felkapcsoltak, a korábban bemutatott eljárások segítségével detektálhatjuk. Mint láttuk, a kapcsoló ki- és bekapcsolt állapotához is egy-egy jól meghatározott digitális érték tartozik.

Sok olyan dolog van azonban, amivel már nem tudunk ilyen egyszerűen megbirkózni, mivel nem csak két jól meghatározott állapota létezhet. Példának okáért gondoljunk a hőmérséklet mérésére. A hőmérő higanyszála is nagyon sok értéket mutathat, ráadásul két jól meghatározott érték között bármilyen értéket felvehet.

Ahhoz, hogy megértsük a folyamatokat, két definícióra van szükségünk:

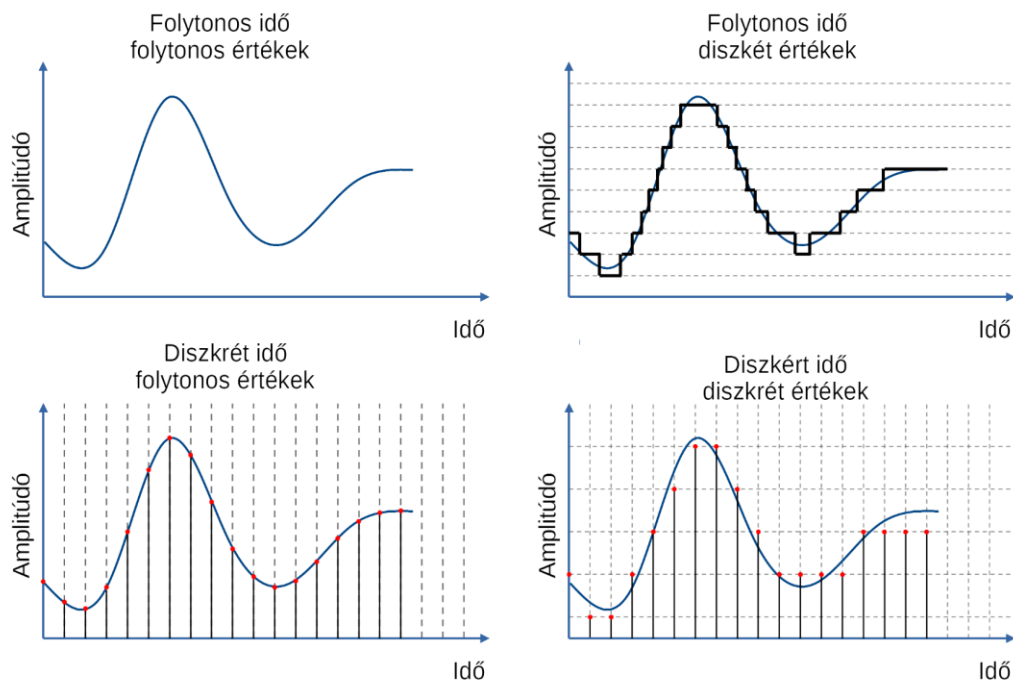
Analóg jel:

Az analóg jel nem más, mint egy valamilyen folytonos fizikai mennyiség (hőmérséklet, páratartalom, nyomás) reprezentációja és az időnek folytonos függvénye. Vegyük alapul a higanyos hőmérőket, ahol a higanyszál hossza (mint analóg jel) mutatja a hőmérsékletet úgy, hogy a két mennyiség egymással arányosan változik.

Mivel ez egy elektronikával foglalkozó tananyag, ezért a legtöbbször előforduló analóg jel a feszültség vagy áram jel, ami az időnek a függvénye.

Digitális jel:

A digitális jel olyan jel, amely nem folytonos időben és csak adott (diszkrét) értékeket vehet fel. Ezt idegen szóval kvantáltnak nevezzük. A digitális jelet általában (leggyakrabban bináris) számokkal kódoljuk. Tehát digitális jelnek azokat a jeleket nevezzük, amelyek csak adott időpillanatokban és csak véges sok, jól megkülönböztethető értéket vesznek fel.

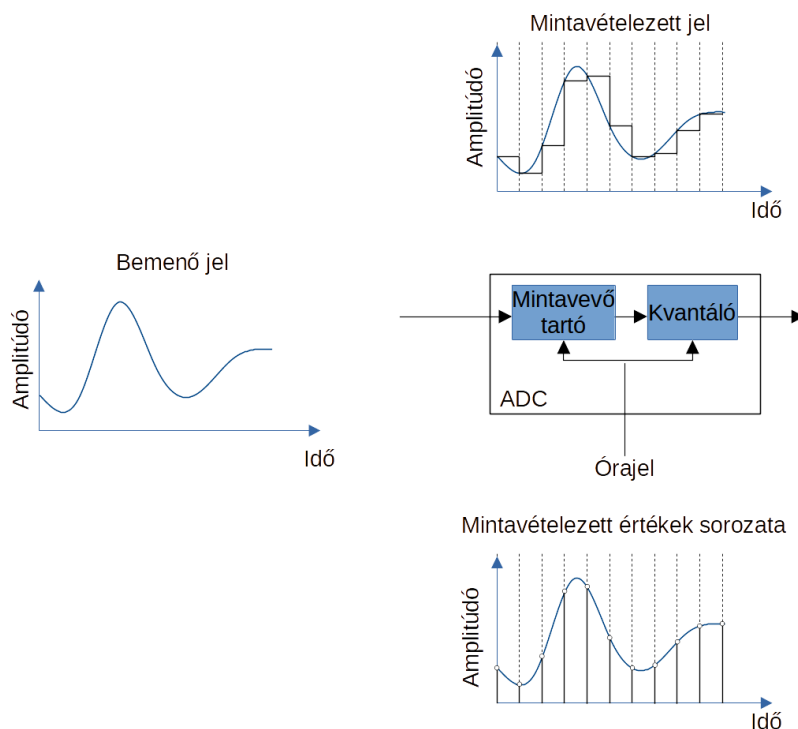


30. ábra: Folytonos és diszkrét jel

Szerencsére képesek vagyunk hőmérsékletet mérni digitális eszközökkel. Ahogy már fentebb is írtuk, a digitális eszközök csak jól definiált (kvantált) értékekkel képesek dolgozni. A feladatunk, hogy a világ analóg jeleit a vezérlő számára is könnyen feldolgozható digitális jelekké alakítsuk.

Erre ad megoldást az analóg-digitális átalakító, röviden ADC (Analog Digital Converter).

Az ilyen eszközöknek rengeteg fajtája létezik, a pontosság, gyorsaság és az átalakítás módjának függvényében. Nekünk most nincs szükségünk mély ismeretekre az átalakítók felépítésével kapcsolatban, elegendő, ha csak a működésük alapjait megértjük.



31. ábra: Analóg-digitális átalakítás folyamata

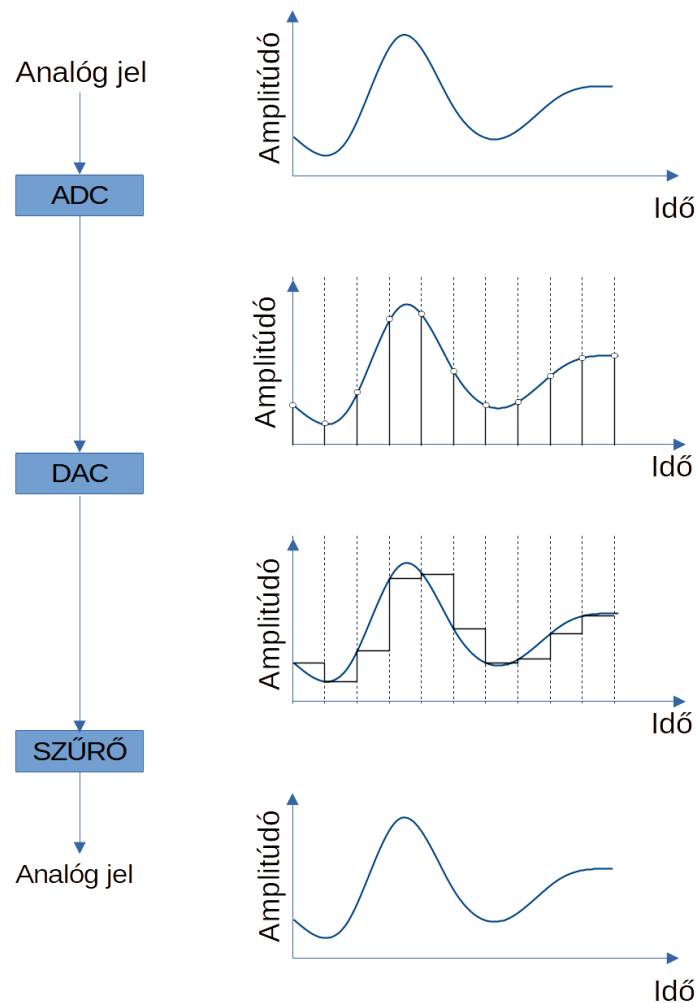
Az A/D átalakító a beérkező jelet képes olyan digitális jellé alakítani, amelyet a vezérlő már könnyen tud kezelni.

DIGITÁLIS-ANALÓG ÁTALAKÍTÓ

Most, hogy már tudjuk, van átjárás az analóg világból a digitális világba, azon gondolkozhatunk, van-e lehetőségünk az egész folyamatot megfordítani. A jó hír, hogy van. Az olyan eszközöket, amelyek egy digitális értékből annak megfelelő analóg jelet állítanak elő, digitális-analóg átalakítónak, DAC-nek (Digital Analog Converter) nevezzük.

Hol találkozhatunk a mindennapi életben ilyen eszközzel? Szinte bárhol. Gondolkoztál már rajta, hogyan tudod kedvenc zenéd meghallgatni a telefonodon? Elárulom, egy ilyen kis eszköz segít benne.

Ebben a tananyagrészen nem szükséges mély ismeretekkel rendelkezni az eszköz működéséről. Elég, ha annyit tudunk, hogy a vezérlővel megadunk egy számot vagy számsorozatot a D/A átalakítónak, az pedig ennek a számnak megfelelő analóg feszültséget állít elő a kimenetén. Ezután még egy analóg szűrőfokozatot is érdemes használni a “szép” kimenő jelalak kedvéért (a jel ne tartalmazzon nagy ugrásokat, meredek fel-, illetve lefutásokat).



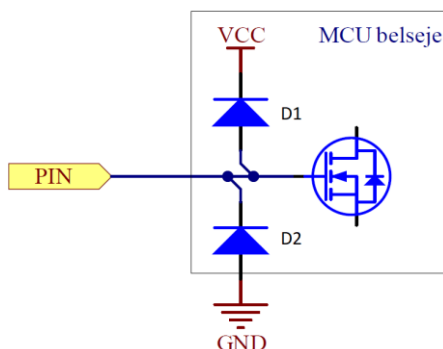
32. ábra: Analóg-Digitális-Analóg átalakítás folyamata

Az ábra kicsit csalóka lehet, mert nem tudjuk a jelünket tökéletesen visszaállítani szűrés után. A különböző mintavételezések és kvantálás torzítást visz a jelbe, ezért a kezdeti ábrához csak nagyon hasonló lesz a visszaállított jel, de nem fog megegyezni vele.

I/O PORTOK VÉDELME

Az elektronikai eszközök esetén kiemelten fontos a működési tartomány betartása, mivel ezen kívül az eszközök könnyen károsodhatnak. Emiatt különböző védelmekkel szükséges ellátnunk őket, amelyek a külső behatások során bekövetkező változásokat szűkítik le a működési tartományba.

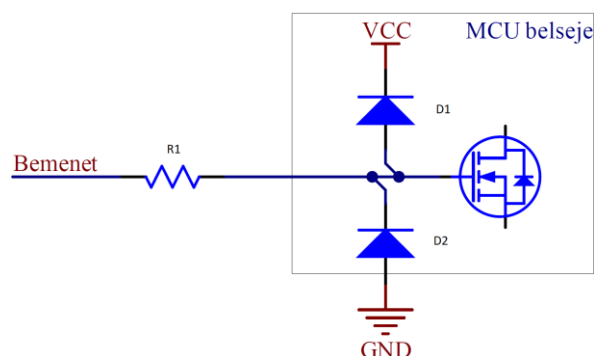
Áramkörünkben a legnagyobb problémákat, akár az eszköz tönkremenetelét is a túlfeszültség és a túláram képes előidézni. A megfelelő védelem érdekében a túlfeszültség és a túláram hatásait vagy magukat a jelenségeket kell megfékezzük.



33. ábra: Mikrovezérlő bemenete a belső diódákkal

A D1 és D2 diódák a mikrovezérlőn belül helyezkednek el és a túlfeszültségvédelemben játszanak szerepet. Amennyiben az adott lábon a feszültség a tápfeszültség és a dióda nyitófeszültségének összege fölé emelkedik, a D1 dióda kinyit és a bemenetet erre az összegfeszültségre korlátozza. Másik esetben, ha bemeneten földpotenciál és a dióda nyitófeszültségének különbsége alá csökken a feszültség, a D2 dióda kinyit és korlátozza a feszültséget.

Ha a túlfeszültség huzamosabb ideig jelen van a bemeneten, akkor a diódákon a disszipált energia mértéke egyre növekszik, a diódák egyre jobban melegednek, végül eléghetnek.

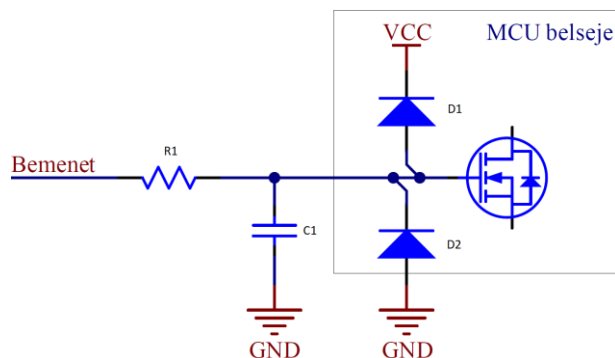


34. ábra: Mikrovezérlő bemenete áramkorlátozó ellenállással

Ezt a problémát egy soros ellenállás beiktatásával orvosolhatjuk. Az ellenállásnak áramkorlátozó szerepe van. Értékét úgy kell megválasztani, hogy a maximális feszültségkülönbség esetén se folyhasson nagyobb áram a belső diódán, mint a megengedett.

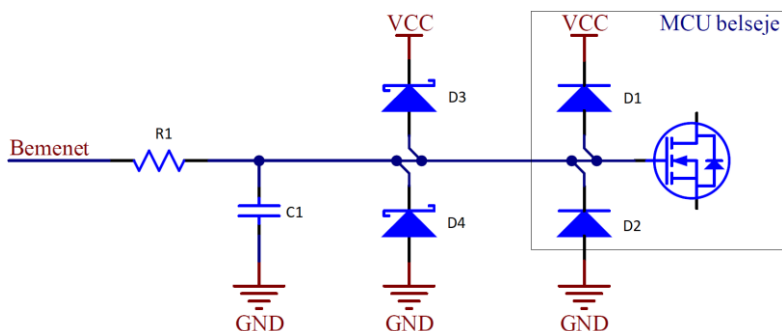
Ezt a megoldást a kimenetek védelmére is felhasználhatjuk. Ekkor úgy kell méretezni, hogy a kontroller adott kimenetét logikai magas vagy alacsony szintre állítva és valamilyen hiba miatt a földhöz vagy tápfeszültséghez rövidzárat feltételezve se folyhasson nagyobb áram, mint az adatlapban megengedett érték.

Egy kondenzátor (C1) beépítésével az R1 ellenállás a C1 kondenzátorral egy RC szűrőt valósít meg. Az ellenállás továbbra is áramkorlátozó szerepet tölt be, míg a kondenzátor szűri a nagyfrekvenciás zavarokat.



35. ábra: Mikrovezérlő bemenete RC szűrővel

Ha nem akarjuk a védelmet a belső diódákra bízni, használhatunk külső, célszerűen Schottky-diódákat (D3 és D4) is. (A Schottky-diódák olyan speciális diódák, amiket egy fém és egy adalékolt szilícium lapka összetételéből alakítanak ki. Gyakorlatban az a nagy előnyük, hogy gyorsabban nyitnak, illetve majdnem fele akkora nyitófeszültséggel rendelkeznek ($\sim 0,4$ V), mint az általános célú diódák.) Így a rendszer redundáns és ezáltal sokkal megbízhatóbb lesz. A védelem akkor fog igazán jól működni, ha úgy választjuk meg a Schottky diódákat, hogy azok kisebb nyitófeszültséggel rendelkeznek, mint a kontroller belsejében található amellet, hogy teljesítményüket is megfelelően nagy értékűnek választjuk. A külső diódák hamarabb kinyitnak, és eldisszipálják a felesleges teljesítményt, ezáltal a D1 és D2 nem fog vezetni, még üzemzavar esetén sem.

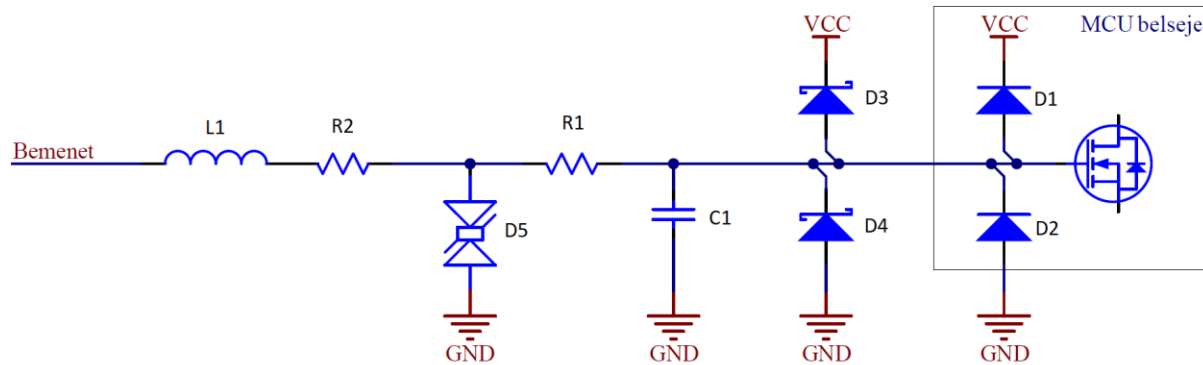


36. ábra: Mikrovezérlő bemenete RC szűrővel és Schottky-diódákkal

Az eszközökben az elektrosztatikai kisülés (ESD - Electrostatic Discharge) által létrehozott pillanatszerű túlfeszültség is káros.

Ilyen esetben TVS (Transient Voltage Suppression) diódákat (D5) kell alkalmazni. Ezek az eszközök gyorsan képesek levágni a nagy feszültségcsúcsokat.

További védelmet jelent a TVS dióda elé helyezett Ferrit-gyűrű (L1) (egy vasmagon elhelyezett tekercs, ezzel növelve az induktivitás nagyságát), ami növeli a tranziens felfutási idejét, ezzel még több időt adva a TVS diódának, hogy működésbe lépjen.



37. ábra: Mikrovezérlő bemenete túláram és túlfeszültség védelemmel

Az esetek többségében szükségtelen és meglehetősen drága minden bemenetet ilyen szintű védelemmel ellátni, de előfordulhatnak olyan biztonságkritikus alkalmazások, ahol elengedhetetlen a felhasználása.

ÖSSZEFOGLALÁS

Ebben a tananyagrészben megismerkedtünk a mikrovezérlő általánosan használható ki- és bemeneti portjaival. Elsajátítottuk, milyen üzemmódokban használhatjuk az adott lábakat és ezek speciális beállításait, mint a kommunikáció, analóg digitális átalakítás vagy éppen a digitális analóg átalakítás.

Végül megtanultuk, hogyan védhetjük meg a vezérlőnket a külvilágból érkező, nem várt hatásoktól, például túlfeszültség (például: hibás bekötés, ESD), túláram.

Remélem ezen tudás birtokában már magabiztosan állsz neki önálló elképzeléseid megvalósításának.